



单位代码 10006

学 号 21374176

分 类 号 TP393.08

北京航空航天大学
BEIHANG UNIVERSITY

毕业设计(论文)

RBA 中基于 Web-WebView-Android 多端关联的
设备身份交叉验证机制研究

学 院 名 称	计算机学院
专 业 名 称	计算机科学与技术
学 生 姓 名	梁 骁
指 导 教 师	巫蜀江

2026 年 6 月



RBA
中基
于
Web
-We
bVie
w-A
ndroi
d 多
端关
联的
设备
身份
交叉
验证
机制
研究

梁

骁

北京
航空
航天
大学



北京航空航天大学

本科生毕业设计(论文)任务书

I、毕业设计(论文)题目:

RBA 中基于 Web-WebView-Android 多端关联的设备身份交叉验证机制研究

II、毕业设计(论文)使用的原始资料(数据)及设计技术要求:

本课题面向移动端 Hybrid App 中的账号安全和无感风控需求, 使用 Android Native、WebView 宿主和 Web 前端运行环境三类设备指纹数据作为原始资料。系统数据包括真实设备采集样本、扩充样本、高危模拟样本、后端合并会话数据、规则知识库标注结果以及消融实验结果。技术上要求完成三端特征采集、统一 session_id 会话对齐、后端数据接收与持久化、跨层语义规则构建、风险标签生成、轻量评分器训练和端侧评分闭环。选题意义在于, 单一 Web 指纹或单一 Native 指纹难以覆盖 Hybrid App 中设备、宿主容器和前端运行环境之间的复杂关系。通过三端融合, 可以利用设备型号、系统版本、屏幕参数、WebView 内核、JSBridge、传感器、WebGL 和 UA 等信息之间的相互印证关系, 提高对模拟器、无头浏览器、接口重放、宿主剥离和云真机环境的识别能力。课题目标是实现一个可运行的原型系统, 并通过实验验证跨层一致性特征对移动端风险判断的有效性。



III、毕业设计（论文）工作内容：

该毕业设计项目的主要工作内容如下：

(1) 调研移动端无感风控、风险自适应认证、浏览器指纹、WebView 安全和端侧推理等相关研究，明确课题研究范围和系统定位；

(2) 设计 HybridGuard 总体架构，划分 Android Native、WebView 宿主、Web 前端、后端服务、规则训练和端侧评分等模块；

(3) 实现三端设备指纹采集与后端会话合并流程，完成原始 JSON 和训练 JSONL 数据持久化；

(4) 构建跨层语义规则知识库，利用规则知识库辅助本地大模型完成离线风险标签生成；

(5) 训练随机森林和 MLP 等轻量评分器，并将随机森林导出为 Android 可运行的 Java 推理代码

(6) 实现端侧评分 App，完成本地三端采集、65 维特征编码、风险分输出和评分摘要上报；

(7) 设计并完成三端特征消融、跨层一致性消融、分组交叉验证和特征重要性分析，撰写毕业论文和答辩材料。

IV、主要参考资料：

[1]Wu S, Sun P, Zhao Y, et al. Him of many faces: characterizing billion-scale adversarial and benign browser fingerprints on commercial websites[C]//NDSS. 2023.



- [2]Sanchez-Rola I, Bilge L, Balzarotti D, et al. Rods with laser beams: Understanding browser fingerprinting on phishing pages[C]//32nd USENIX Security Symposium (USENIX Security 23). 2023: 4157-4173.
- [3]Snyder P, Karami S, Edelstein A, et al. {Pool-Party}: Exploiting browser resource pools for web tracking[C]//32nd USENIX Security Symposium (USENIX Security 23). 2023: 7091-7105.
- [4]Cao Y, Li S, Wijmans E. (Cross-) browser fingerprinting via OS and hardware level features[C]//Proceedings 2017 Network and Distributed System Security Symposium. Internet Society, 2017.
- [5]Vastel A, Laperdrix P, Rudametkin W, et al. {Fp-Scanner}: The privacy implications of browser fingerprint inconsistencies[C]//27th USENIX Security Symposium (USENIX Security 18). 2018: 135-150.
- [6]Jiang Y, Zhu H, Chang S, et al. MAUTH: Continuous user authentication based on subtle intrinsic muscular tremors[J]. IEEE Transactions on Mobile Computing, 2023, 23(2): 1930-1941.
- [7]Tiwari A, Prakash J, Gross S, et al. A large scale analysis of android—web hybridization[J]. Journal of Systems and Software, 2020, 170: 110775.
- [8]Sun Z, Sun R, Liu C, et al. Shadownet: A secure and efficient on-device model inference system for convolutional neural networks[C]//2023 IEEE Symposium on Security and Privacy (SP). IEEE, 2023: 1596-1612.



学生 梁骁

毕业设计(论文)时间: 2025 年 12 月 24 日至 2026 年 6 月 4 日

答辩时间: 2026 年 6 月 4 日

成 绩: _____

指导教师: _____

兼职教师或答疑教师(并指出所负责部分):

_____ 6 _____ 系(教研室) 主任(签字): _____

注: 任务书应该附在已完成的毕业设计(论文)的首页



本人声明

我声明,本论文及其研究工作是由本人在导师指导下独立完成的,在完成论文时所利用的一切资料均已在参考文献中列出。

作者: 梁骁

签字:

时间: 2026 年 6 月



RBA 中基于 Web-WebView-Android 多端关联的设备身份交叉验证机制研究

学 生：梁 骁

指导教师：巫蜀江

摘 要

随着移动互联网应用承载登录、支付和业务办理等操作，账号安全已从口令校验扩展为对访问环境的持续评估。风险自适应认证（RBA）通过综合设备、环境和上下文信息判断访问风险，在低风险场景减少打扰，在高风险场景触发更强认证。但在 Hybrid App 中，业务会话同时经过 Android 原生层、WebView 宿主和 Web 前端。若只依赖单一设备指纹，容易被脚本伪造、无头浏览器、模拟器或接口重放绕过；若完全依赖服务端风控，又会带来延迟和隐私压力。

针对上述问题，本文设计并实现 HybridGuard 系统，在同一会话中采集三类运行环境信息，通过会话标识完成异步对齐，并围绕设备身份、屏幕参数、浏览器标识和调试状态等特征建立可解释的校验关系。系统将离线规则标注结果训练为轻量评分器，使 App 能够本地输出风险分。实验结果表明，本文方法能够把分散指纹转化为稳定的风险判断依据。在严格分组交叉验证下，少量核心校验特征取得优于完整原始字段拼接的误差表现，说明信息互证比简单堆叠字段更有利于泛化，为移动端 RBA 提供了兼顾安全性、可用性和隐私约束的可落地方案。

关键词： 风险自适应认证，Android，WebView，三端设备指纹，跨层一致性，端侧评分



Research on a Device Identity Cross-Verification Mechanism Based on Web-WebView-Android Multi-End Association in RBA

Author : LIANG Xiao

Tutor : WU Shu-jiang

Abstract

As mobile Internet apps increasingly support login, payment, and business transactions, account security has shifted from password checks to continuous assessment of the access environment. Risk-Based Authentication (RBA) evaluates risk using device, environment, and contextual information, reducing interruption in low-risk scenarios while triggering stronger authentication for suspicious access. However, in Hybrid Apps, a session spans the Android native layer, WebView host, and Web frontend. A single device fingerprint can be bypassed by forged scripts, headless browsers, emulators, or API replay, while server-side risk control introduces latency and privacy pressure.

To address these issues, this thesis designs and implements HybridGuard, which collects runtime information from the three environments in the same session, aligns asynchronous data by session identifier, and builds interpretable checks based on device identity, screen parameters, browser identifiers, and debugging status. A lightweight scorer trained from offline rule-based labels enables local risk scoring in the app. Experiments show that dispersed fingerprints can be turned into stable risk evidence. Under strict grouped cross-validation, a small set of core verification features achieves lower error than all raw fields, showing that cross-source verification generalizes better than simple feature stacking.

Key words: Risk-based authentication, Android, WebView, Tri-end device fingerprinting, Cross-layer consistency, On-device scoring



目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 设备指纹与浏览器指纹研究	2
1.2.2 移动端风控与无感认证研究	3
1.2.3 Hybrid App、WebView 与 JSBridge 安全研究	3
1.2.4 端侧评分与轻量模型研究	4
1.2.5 相关研究总结	4
1.3 研究目标与研究内容	5
1.4 论文组织结构	6
2 系统设计与总体架构	7
2.1 系统设计目标	7
2.2 功能需求分析	8
2.2.1 跨端设备指纹采集功能	8
2.2.2 会话合并与数据持久化功能	8
2.2.3 规则知识库与风险标签生成功能	9
2.2.4 轻量评分与 App 本地评分功能	9
2.2.5 实验分析功能	9
2.3 非功能需求分析	9
2.3.1 识别准确性与鲁棒性	9
2.3.2 端侧性能与资源占用	10
2.3.3 隐私与数据最小化	10
2.3.4 可维护性与可扩展性	10
2.4 系统总体架构设计	10
2.5 系统工作流程	11



2.5.1	离线标注训练链路	11
2.5.2	端侧评分训练链路	12
2.5.3	跨层一致性分析流程	13
2.6	本章小结	14
3	关键模块设计与实现	15
3.1	跨端设备指纹采集模块设计与实现	15
3.1.1	Android Native 原生特征采集	15
3.1.2	WebView 宿主容器特征采集	16
3.1.3	Web 前端运行环境特征采集	16
3.1.4	端侧评分 App 中的本地采集	17
3.2	后端数据接收、合并与持久化模块	17
3.2.1	Pydantic 数据模型	17
3.2.2	会话合并策略	18
3.2.3	训练数据展开	18
3.3	跨层语义规则知识库与风险标签生成模块	18
3.3.1	规则知识库构建思路	18
3.3.2	大模型辅助风险评分流程	19
3.3.3	标签质量控制	20
3.3.4	规则示例	21
3.4	数据集构建与样本生成模块	21
3.4.1	真实设备数据采集	21
3.4.2	真实样本扩充	21
3.4.3	高危样本生成	22
3.4.4	当前数据规模	22
3.5	轻量风险评分模块	22
3.5.1	特征工程	22
3.5.2	候选评分器实现	23
3.5.3	端侧部署模型选择依据	23



3.6	App 本地评分模块设计与实现	23
3.6.1	端侧模块结构	23
3.6.2	端侧特征编码	24
3.6.3	本地随机森林推理与结果上报	24
3.6.4	工程风险与处理	25
3.7	本章小结	25
4	实验设计与结果分析	26
4.1	实验环境与数据集说明	26
4.1.1	实验环境	26
4.1.2	数据来源与样本规模	26
4.1.3	风险标签与分数分布	27
4.1.4	三端采集字段完整性统计	27
4.2	实验目标、实验设计与评估指标	28
4.2.1	实验目标	28
4.2.2	评估指标	28
4.2.3	数据划分方式	28
4.3	粗粒度三端特征消融实验	29
4.3.1	实验目的与分组	29
4.3.2	随机 Holdout 结果与分析	30
4.3.3	分组交叉验证结果与分析	30
4.4	跨层一致性特征构造与消融实验	31
4.4.1	实验目的	31
4.4.2	一致性特征构造方法	31
4.4.3	一致性消融实验分组	32
4.4.4	随机 Holdout 结果与分析	32
4.5	分组交叉验证下的跨层一致性泛化分析	33
4.5.1	实验目的	33
4.5.2	分组版一致性消融结果	33



4.5.3 结果分析	34
4.6 特征重要性与跨层规则解释	34
4.6.1 实验目的	34
4.6.2 重要特征结果	35
4.6.3 规则解释与系统贡献对应关系	35
4.7 本章讨论与局限性	36
4.7.1 实验结论汇总	36
4.7.2 局限性说明	36
4.7.3 后续改进方向	37
4.8 本章小结	37
结论	38
致谢	41
参考文献	42



1 绪论

1.1 研究背景与意义

随着移动互联网应用逐渐承载登录、支付、社交、内容发布和业务办理等高价值操作,移动端账号安全与业务风控的重要性不断提升。业务系统在验证账号身份之外,还需要判断当前访问是否来自可信设备、可信宿主和真实用户环境。传统口令、短信验证码、图形验证码等显式认证方式虽然能够提高攻击成本,但可能中断用户操作流程,影响低风险场景下的使用体验。因此,如何在尽可能减少用户交互负担的情况下识别异常设备、自动化脚本、模拟器、云真机和接口重放行为,成为移动端无感风控中的重要问题。

风险自适应认证 (Risk-Based Authentication, RBA) 通过综合设备、环境、行为和上下文信息估计访问风险,并据此决定是否放行或触发更强认证。相关研究指出,移动端认证需要在安全性与可用性之间取得平衡,不能对所有访问请求无差别地增加显式验证强度^[1]。也有研究进一步尝试使用强化学习动态选择认证策略,说明风险评分与认证动作之间可以形成自适应决策闭环^[2]。设备指纹正是风险自适应认证中的一种重要无感信号,它通过组合设备硬件、操作系统、浏览器运行环境、应用容器和网络上下文等属性,辅助系统识别可信设备和异常环境。

在 Web 场景中, User-Agent、屏幕参数、时区、Canvas、WebGL、硬件并发数等特征已经被广泛用于浏览器指纹构建。浏览器指纹采集系统研究对字段组织、脚本探针和数据集构建进行了整理,为本文 Web 端探针设计提供了参考^[3]。已有研究对真实商业网站中的大规模浏览器指纹进行了分析,说明浏览器指纹可用于区分普通访问与对抗性访问^[4]。同时,浏览器指纹也可能被钓鱼页面和攻击者利用,具有安全防护与隐私风险并存的特点^[5]。因此,W3C 等组织也将浏览器指纹列为 Web 隐私治理中的重要问题^[6]。除常见显式字段外,浏览器资源池等内部机制也可能形成跨站跟踪信道,说明 Web 运行环境的可观测面仍在持续扩展^[7]。

移动端 Hybrid App 场景比普通 Web 场景更加复杂。一个业务会话通常同时涉及 A



ndroid Native 原生层、WebView 宿主容器和 Web 前端运行环境。Native 层可以采集设备型号、系统版本、Build 信息、内存、物理屏幕、电池、传感器和调试状态；WebView 层可以反映宿主容器、JSBridge、WebView 内核、安装来源和 App 配置；Web 层则暴露 Navigator、逻辑屏幕、DPR、WebGL、Canvas、时区和算力挑战等特征。三类特征在同一设备会话中形成可相互印证的关系。例如，Native 物理屏幕应与 Web 逻辑屏幕和 DPR 近似对应，WebView provider 版本应与 Web User-Agent 中的浏览器内核版本相互呼应，JSBridge 是否存在能够反映页面是否运行在预期 App 宿主中。

单一来源的设备指纹难以充分覆盖上述复杂环境。只依赖 Web 指纹时，攻击者可以通过自动化浏览器、无头环境或脚本代理伪造部分字段；只依赖 Native 指纹时，系统难以观察 WebView 运行时和前端渲染环境；仅依赖 WebView 宿主信息时，则可能忽略设备物理属性与 Web 暴露信息之间的矛盾。尤其在模拟器、云真机、调试包滥用和接口重放等场景下，局部字段可以被伪造，但多层特征之间的一致性更难长期保持。

基于上述背景，本文设计并实现一种面向 Hybrid App 的三端融合设备指纹无感风控系统 HybridGuard。本文所称“三端”是指同一移动端会话中的 Android Native、WebView 宿主和 Web 前端三类特征来源。系统通过统一 session_id 完成三端异步采集结果对齐，并进一步分析跨层语义一致性关系，形成风险规则、离线标签和端侧轻量评分能力。本文的意义在于：一是贯通同一会话下三类原本分离的指纹源，形成更完整的移动端环境视图；二是从单字段判断转向跨层一致性分析，提高对伪造环境和宿主剥离行为的解释能力；三是将离线规则分析结果压缩为端侧评分器，使 App 能够在本地完成风险分输出，减少对远程风控服务和原始指纹上传的依赖^{[8][9]}。

1.2 国内外研究现状

1.2.1 设备指纹与浏览器指纹研究

设备指纹研究最早在 Web 跟踪、反欺诈和访问控制等场景中受到关注。浏览器指纹通过组合 User-Agent、屏幕、时区、语言、字体、Canvas、WebGL 等多维属性，在不依赖传统 Cookie 的情况下刻画浏览器或设备环境^[10]。近年来，研究者不仅关注浏览器指纹的唯一性，也关注指纹脚本检测、隐私保护和对抗性伪造等问题。FP-Fed 研究了隐私保护的浏览器指纹检测方法^[8]，FP-tracer 则从 API 调用、污点传播和熵阈值角度识别细粒度指纹行为^[11]。



与本文更相关的是,部分研究开始关注指纹字段之间的结构关系和一致性问题。BrowserFinger 通过特征模型分析浏览器指纹与浏览器、操作系统、硬件配置之间的映射关系,说明 Web 侧暴露信息与底层系统环境存在联系^[12]。早期跨浏览器指纹研究也表明,操作系统和硬件层特征能够跨越不同浏览器形成识别信号^[13]。远程 GPU 指纹研究进一步说明, WebGL 等渲染能力不仅反映浏览器状态,也可能暴露硬件栈差异^[14]。此外, Fingerprint Scanner 指出浏览器指纹中的字段不一致可能暴露伪造、自动化或隐私插件造成的异常环境^[15]。这些研究为本文从“采集字段值”转向“分析字段关系”提供了重要启发。

1.2.2 移动端风控与无感认证研究

移动端认证研究长期关注安全性与可用性的平衡。无感认证和连续认证尝试利用移动设备在自然使用过程中产生的传感器、触摸、姿态和上下文数据,对用户或设备状态进行持续评估^[16]。已有研究利用肌肉震颤、触控屏特征、触摸事件和人体活动等信号实现智能手机上的隐式认证^{[17][18][19]}。也有研究将触摸手势和按键动态进行特征级融合,以缓解单一行为特征在安全性和可用性之间的权衡问题^[20]。这些工作说明,移动设备能够提供丰富的低干扰风险信号,适合用于无感安全判断。

从设备环境角度看,Android 原生系统暴露的品牌、型号、系统版本、Build Fingerprint、CPU ABI、内存、屏幕、电池、传感器、调试状态和安装来源等信息,也能够反映设备真实性和运行环境稳定性。已有研究关注 Android App 中设备指纹活动的检测,说明移动应用内部确实存在对设备属性进行组合采集和识别的实践^[21]。同时,传感器和硬件行为也可用于手机型号识别或用户认证,进一步说明物理环境特征具有安全判断价值^[22]。

不过,现有移动端风控研究多数集中在单一模态或单一层级,例如用户行为、传感器、浏览器指纹或 Android 安全检测。对于 Hybrid App 而言,风险往往出现在 Native、WebView 和 Web 前端之间的边界处。单独分析某一层容易遗漏跨层矛盾,因此需要将多层设备指纹放在同一会话中进行融合判断。

1.2.3 Hybrid App、WebView 与 JSBridge 安全研究

Hybrid App 通过 WebView 将 Web 页面嵌入 Android 原生应用,使业务可以同时利用 Web 前端的灵活迭代能力和 Native 层的设备能力。已有大规模分析表明,Android



d 应用中 WebView 和 Android-Web 混合化机制使用广泛, 同时也会带来 JavaScript 与原生能力交互的安全问题^[23]。WebView 通常允许 JavaScript 与 Android 原生代码通过 JSBridge 进行双向通信。Android 官方文档提供了 WebView、JavaScript 接口和本地内容加载等机制, 同时也提示不安全的 native bridge 可能带来攻击风险^{[24][27]}。

相关研究表明, Android 与 Web 混合应用中的 Java/JavaScript 交互会引入复杂的信息流和权限边界问题。Demand-driven Information Flow Analysis of WebView in Android Hybrid Apps 研究了 WebView 中面向需求的信息流分析方法, 说明传统单端分析难以完整捕获 Hybrid App 中跨语言、跨层级的数据流^[28]。关于 Android WebView 对象漏洞的研究也指出, WebViewClient、JavaScript interface 和 WebView 配置不当可能导致敏感能力暴露和跨边界攻击^[29]。

已有 WebView 研究多从漏洞检测、信息流分析和安全配置角度展开, 较少将 WebView 宿主环境作为无感风控中的独立指纹层。本文将 WebView 宿主与 Android Native、Web 前端并列建模, 采集 WebView provider、默认 User-Agent、JSBridge 注入状态、安装来源、调试状态等特征, 并将其用于跨层一致性校验。这样, WebView 不仅是页面承载容器, 也成为连接 Native 与 Web 的可信宿主证明来源。

1.2.4 端侧评分与轻量模型研究

传统风控系统通常将设备指纹上传到服务端, 由服务端完成规则匹配和模型推理。这种架构便于集中管理, 但也带来延迟、网络依赖、隐私压力和服务端成本。随着移动端算力提升, 越来越多安全与智能任务开始考虑在终端本地执行部分推理。端侧推理能够降低网络往返延迟、提高离线可用性, 并减少原始数据离开设备的必要性, 但需要控制模型体积、推理耗时、内存占用和模型保护问题^{[9][30]}。移动端机器学习系统还需要面对内存限制、设备异构性和不同推理框架带来的延迟差异^{[31][32]}。

TinyML 和边缘推理研究表明, 在资源受限设备上部署轻量模型需要综合考虑计算开销、存储开销、精度和可解释性^[33]。本文面向结构化设备指纹和一致性特征, 采用随机森林等轻量模型进行工程对比, 并通过轻量评分器压缩离线规则分析和标签生成结果, 形成端侧风险评分闭环。

1.2.5 相关研究总结



综上, 现有研究已经在浏览器指纹、移动端无感认证、Android 安全分析、WebView 安全和端侧推理等方面形成了较多成果。浏览器指纹研究证明 Web 运行环境中多种 API 和系统属性能够组合形成识别能力; 移动端认证研究说明传感器、触摸和上下文数据可以支持低打扰认证; WebView 安全研究揭示了 Hybrid App 中 Native 与 Web 边界的复杂性; 端侧推理研究则说明将部分智能决策放到终端本地具有降低延迟和保护隐私的价值。

但面向 Hybrid App 无感风控, 现有研究仍有不足。第一, 许多工作集中于单端特征, 缺少将 Native、WebView 和 Web 前端作为同一会话内互证信号的系统方案。第二, 部分方法重视采集字段数量, 但没有显式建模不同层级字段之间的语义一致性。第三, WebView 研究多关注漏洞与信息流, 较少将宿主容器作为设备指纹和风控判断的关键来源。第四, 服务端集中评分对网络、延迟和隐私提出更高要求, 缺少面向 App 本地评分闭环的轻量实现。基于这些不足, 本文提出并实现三端融合设备指纹系统 Hybrid Guard, 重点研究同一移动端会话下三端特征的采集、对齐、语义建模和端侧风险评分。

1.3 研究目标与研究内容

本文的总体目标是设计并实现一种面向 Hybrid App 场景的三端融合设备指纹无感风控系统。系统需要在同一移动端会话中采集 Android Native、WebView 宿主和 Web 前端三类特征, 对异步采集结果进行会话合并, 并通过跨层语义一致性规则识别模拟器、无头环境、接口重放、宿主剥离和异常调试环境等风险。最终, 系统将离线规则分析和风险标签生成能力压缩为端侧轻量评分器, 使 App 能够在本地输出风险分。

围绕上述目标, 本文主要研究内容如下。

设计三端设备指纹采集模型。Native 侧采集设备构建、系统版本、内存、物理屏幕、电池、传感器和安全配置等底层特征; WebView 侧采集 JSBridge、WebView provider、默认 User-Agent、安装来源和 SDK 配置等容器特征; Web 侧采集 Navigator、逻辑屏幕、DPR、WebGL、Canvas、算力挑战和时区等运行环境特征。

实现会话合并与数据持久化流程。系统以 session_id 为主键对三端异步采集结果进行增量合并, 保存原始嵌套结构, 并生成面向训练和分析的 JSONL 数据, 为后续风险标签生成和模型训练提供可复现数据链路。

构建跨层语义规则知识库。本文利用大语言模型辅助分析不同层级特征之间的语义



对应关系,归纳形成屏幕一致性、UA 一致性、WebView 内核一致性、JSBridge 宿主真实性、传感器物理可信性和 WebGL 渲染环境一致性等规则。

构建风险评分数据集与离线标注流程。系统采集真实设备样本,并结合扩充样本和高危模拟样本形成实验数据集。离线阶段根据规则知识库调用本地大模型生成 risk_score 和 risk_reason,再将带标签数据用于训练轻量评分器。

实现端侧轻量评分闭环。本文将训练得到的轻量评分器导出为 Android 可运行代码,并在 App 中完成本地特征编码、模型推理和风险结果展示。端侧运行时,系统仅上传风险摘要信息,减少原始三端指纹外传需求。

设计实验验证系统有效性。实验从三端字段完整性、粗粒度特征消融、跨层一致性消融、分组交叉验证和特征重要性分析等角度展开,重点验证三端融合与跨层语义一致性建模对移动端风险判断的贡献。

1.4 论文组织结构

本文围绕三端融合设备指纹无感风控系统的设计、实现与验证展开,全文结构如下。

第 1 章为绪论,介绍移动端无感风控、风险自适应认证和设备指纹技术的研究背景,分析单端指纹在 Hybrid App 场景下的局限,总结相关研究现状,并给出本文研究目标和论文组织结构。

第 2 章为系统设计与总体架构,介绍 HybridGuard 的系统需求、设计目标、总体架构和 workflows,重点说明三端特征源划分、会话合并、规则知识库、风险评分和端侧决策之间的关系。

第 3 章为关键模块设计与实现,介绍三端设备指纹采集、后端数据接收与持久化、跨层语义规则知识库、数据集构建、轻量评分器训练和 Android 端侧评分 App 的实现细节。

第 4 章为实验设计与结果分析,介绍实验环境、数据来源、风险标签分布和三端字段完整性,并通过三端特征消融、跨层一致性消融、分组交叉验证和特征重要性分析评估系统有效性。

最后为结论与展望,总结本文完成的主要工作和实验结论,分析当前系统在数据规模、真实攻击样本、多设备覆盖和规则自动更新等方面的不足,并展望后续改进方向。



2 系统设计与总体架构

2.1 系统设计目标

HybridGuard 面向移动端 Hybrid App 场景,目标是在不显著增加用户交互负担的前提下,对设备环境、App 宿主和 Web 运行时进行联合刻画,并输出可用于风控决策的风险分。与只采集单端字段的设备指纹系统不同,本文关注同一移动端会话中 Android Native、WebView 宿主容器和 Web 前端运行环境之间的语义呼应关系。系统设计目标主要包括以下几个方面。

第一,三端融合一致性。系统需要同时采集 Native、WebView 和 Web 三类特征,并通过统一会话标识进行对齐,使同一设备在不同层级暴露出的设备型号、系统版本、屏幕参数、浏览器内核、JSBridge 和渲染环境等信息能够被联合分析。三端融合主要关注不同层级特征之间的相互印证关系。

第二,无感采集。系统采集过程应尽量在 App 正常启动和 WebView 页面加载过程中自动完成,不要求用户额外交互。移动端无感风控的价值在于对低风险访问保持较低打扰,而在高风险环境中为后续认证或拦截策略提供依据。

第三,语义规则化。单字段阈值难以覆盖复杂移动端环境。系统需要将 Native 物理屏幕与 Web DPR、Native Build 信息与 Web User-Agent、WebView provider 与 Web 浏览器内核、JSBridge 与 App 宿主真实性等关系抽象为跨层语义规则。规则知识库用于表达不同层级特征之间的可解释关系,并为后续风险标签生成和轻量评分器训练提供依据。大语言模型在网络安全分析中的应用研究表明,LLM 可用于辅助安全语义分析和规则整理,但在线直接决策仍需考虑成本、稳定性和可控性^[34]。

第四,端侧闭环。系统应将离线规则分析和标注结果压缩为端侧轻量评分器,避免长期依赖远程服务完成所有风险判断。端侧推理有助于降低网络延迟、增强离线可用性,并减少原始指纹数据离开设备的需求^[9]。因此,HybridGuard 需要支持在 Android 端完成三端本地采集、特征编码、模型推理和风险摘要上报。

第五,可扩展与可复现。系统需要在采集字段、后端数据模型、训练脚本和端侧编码之间保持一致,便于后续增加行为特征、网络特征或替换评分模型。同时,数据采集、



标签生成、训练评估和消融实验应能独立执行，保证实验结果可复现。

2.2 功能需求分析

2.2.1 跨端设备指纹采集功能

系统首先需要完成三类设备指纹采集。Android Native 侧负责采集底层设备与系统特征，包括设备品牌、型号、系统版本、API Level、CPU ABI、Build Fingerprint、内存、物理屏幕、电池状态、传感器矩阵和 ADB 调试状态等。WebView 宿主侧负责采集容器相关特征，包括 JSBridge 注入状态、WebView provider、默认 User-Agent、App 安装来源、安装更新时间、target SDK 和调试配置等。Web 前端侧负责采集运行时特征，包括 Navigator、逻辑屏幕、DPR、WebGL vendor/renderer、Canvas 哈希、算力挑战、时区和语言等。

三类采集源的设计关系如表 2.1 所示。

表 2.1 三类采集源设计

特征来源	主要采集内容	风控意义
Android Native	设备型号、系统版本、Build、屏幕、电池、传感器、ADB	描述底层设备真实性和系统环境
WebView 宿主	JSBridge、WebView provider、宿主 UA、安装来源、SDK 配置	判断页面是否运行在预期 App 容器内
Web 前端	Navigator、逻辑屏幕、DPR、WebGL、Canvas、时区、算力	描述网页脚本实际看到的运行环境

WebView 与 JavaScript 之间的桥接能力是 Hybrid App 的关键机制，但不安全的 native bridge 也可能带来攻击风险，因此系统在设计中既利用 JSBridge 作为宿主真实性信号，也将其纳入风险分析范围^{[24][27]}。

2.2.2 会话合并与数据持久化功能

三端采集数据并不总是在同一时刻产生。Native 层通常可以在 App 启动后较早完成采集，WebView 和 Web 前端数据则依赖页面加载、JavaScript 执行和 JSBridge 回调。因此，系统需要以 session_id 为主键对异步上报数据进行增量合并。每次上报到达后，后端根据会话标识查找已有记录，将非空字段合并到同一会话对象中，并保留原始嵌套结构。

数据持久化需要同时服务两类目标：一是保存完整原始会话，便于回溯采集结果和



分析异常样本；二是生成面向训练和评估的扁平化 JSONL 数据，便于后续规则标注、特征工程和模型训练。系统因此需要同时支持原始 JSON 存储和训练数据展开。

2.2.3 规则知识库与风险标签生成功能

系统需要根据三端特征之间的语义关系构建规则知识库。规则不只关注单个字段是否异常，而是关注跨层信息是否一致。例如，Native 层物理屏幕尺寸与 Web 层逻辑屏幕乘以 DPR 是否近似一致；Native 设备型号和 Android 版本是否能与 Web User-Agent 呼应；WebView provider 版本是否与 Web 层浏览器内核版本一致；JSBridge 是否存在并能返回同一会话信息；传感器、电池和 WebGL 渲染环境是否符合移动真机特征。

在离线阶段，系统利用规则知识库辅助本地大模型对样本生成 risk_score 和 risk_reason。该流程可将复杂的跨层语义分析转化为带标签数据，为后续轻量评分器训练提供监督信号。该流程中的 LLM 主要服务于离线规则解释和标签生成，端侧在线阶段由轻量评分器输出风险结果。

2.2.4 轻量评分与 App 本地评分功能

端侧评分功能要求系统在 App 内完成本地采集、特征向量构造和模型推理。训练阶段生成的特征顺序、类别编码和缺失值处理策略需要固化到 Android 端，避免训练侧与端侧输入不一致。评分器输出风险分、风险等级、风险原因和评分引擎信息后，App 只需上传评分摘要，原始三端指纹不必在端侧评分链路中持续外传。对于资源受限设备，轻量模型部署需要兼顾模型规模、推理时间和可解释性^[33]。

2.2.5 实验分析功能

系统还需要支持实验分析，包括三端采集完整性统计、风险标签分布分析、轻量评分器误差评估、三端特征消融、跨层一致性消融、分组交叉验证和特征重要性分析。实验分析功能用于回答本文的核心问题：三端特征是否能形成互证关系，跨层一致性是否是有效风险信号，轻量评分器是否能在端侧复现离线风险判断能力。

2.3 非功能需求分析

2.3.1 识别准确性与鲁棒性

系统应能够识别正常真机、模拟器、云真机、调试环境、无头浏览器和接口重放等



不同风险场景。由于攻击者可能局部伪造某一层字段，系统设计不依赖单个强特征，而是通过多层特征之间的语义一致性降低单点伪造带来的误判风险。

2.3.2 端侧性能与资源占用

端侧评分需要控制采集耗时、WebView 加载开销、特征编码耗时、模型体积和推理耗时，避免影响 App 启动和页面加载体验。模型选择围绕端侧部署可行性展开，随机森林、MLP 等轻量模型用于平衡推理开销、模型体积和可解释性。

2.3.3 隐私与数据最小化

设备指纹具有隐私敏感性。Web 标准和浏览器隐私治理均关注指纹技术可能导致的跨站识别和用户追踪风险^[6]。因此，本文系统设计遵循数据最小化原则：采集内容聚焦于设备环境一致性验证，不采集通讯录、文件、短信等用户私密内容；端侧评分链路尽量只上传风险分、风险等级和评分摘要，减少原始三端指纹长期外传。

2.3.4 可维护性与可扩展性

系统中采集字段、后端数据模型、离线训练脚本和端侧特征编码存在强一致性要求。任何字段新增、删除或编码方式变化，都可能导致训练侧与端侧推理输入不一致。因此，系统需要保持明确的字段分层和特征版本管理。后续若加入行为特征、网络特征或更多 Web API 探针，也应沿用三端分层和会话合并机制。

2.4 系统总体架构设计

HybridGuard 采用分层架构，整体包括采集层、服务与数据层、规则与训练层、端侧评分层四个部分。采集层负责从 Android Native、WebView 宿主和 Web 前端获取原始指纹；服务与数据层负责接收数据、合并会话和持久化；规则与训练层负责构建跨层语义规则、生成风险标签并训练轻量评分器；端侧评分层负责在 App 内完成本地采集、特征编码和风险分输出。

系统总体架构如图 2.1 所示。

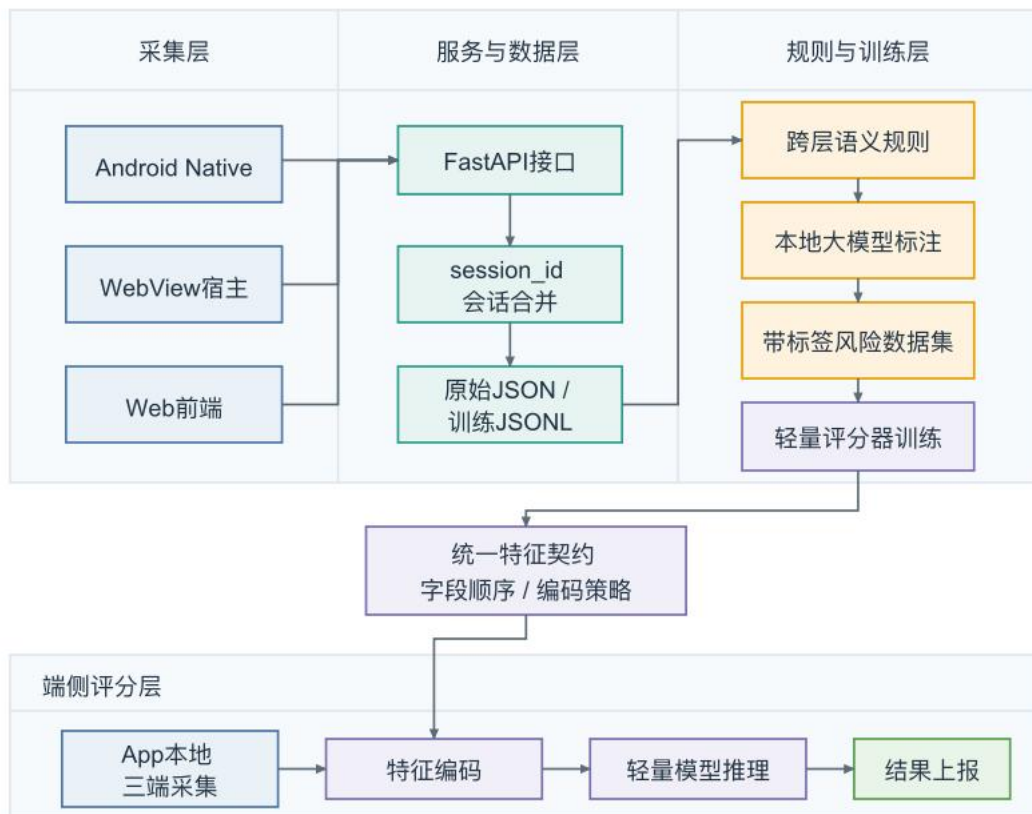


图 2.1 系统总体架构：离线训练与端侧评分双链路

图 2.1 中，上半部分主要对应离线采集、标注和训练链路，下半部分对应端侧评分运行链路。两条链路通过统一的特征定义和编码策略连接：离线训练阶段确定字段顺序、类别编码和缺失值处理方式，端侧运行阶段必须按相同规则构造输入向量。这样才能保证端侧评分结果与离线评估结果具有一致含义。

从数据组织角度看，系统将三端特征保留为相对独立的命名空间。Native 字段用于描述底层系统和物理环境，WebView 字段用于描述 App 宿主容器，Web 字段用于描述前端脚本观察到的运行环境。跨层语义规则则位于三类原始字段之上，负责把“字段值”转化为“字段关系”。例如，屏幕一致性规则需要同时读取 Native 物理屏幕、Web 逻辑屏幕和 DPR；UA 一致性规则需要同时读取 Native Build、WebView 默认 UA 和 Web Navigator UA；宿主真实性规则需要结合 JSBridge 注入状态和会话一致性。

2.5 系统工作流程

HybridGuard 的工作流程分为离线标注训练链路和端侧评分运行链路。前者用于采集数据、构建规则、生成标签和训练模型；后者用于在 App 内完成本地风险评分。

2.5.1 离线标注训练链路

离线链路从旧采集 App 开始。App 启动后生成 session_id，Native 层采集设备原生特征并上报后端；随后 WebView 加载前端探针页面，JSBridge 返回 WebView 宿主特征，Web 前端脚本采集浏览器运行环境、WebGL、Canvas 和算力挑战等特征。后端按 session_id 合并三端数据并写入持久化文件。随后，系统基于跨层语义规则知识库和本地大模型生成风险标签，再将带标签数据用于轻量评分器训练和消融实验。

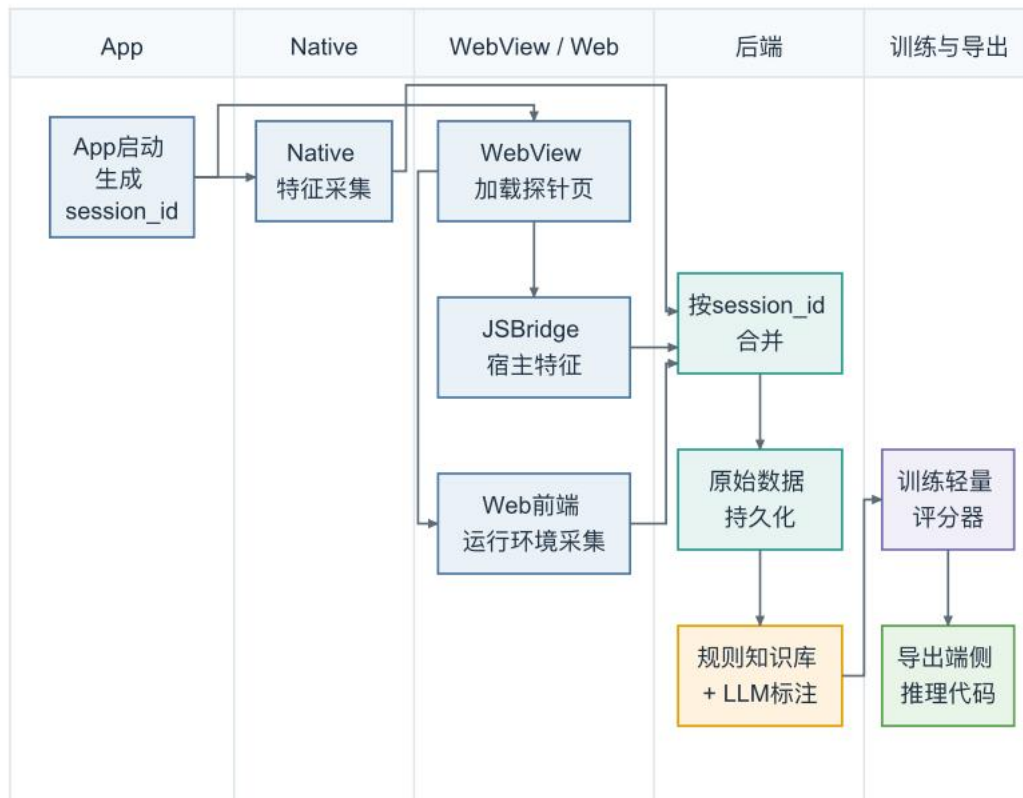


图 2.2 离线标注训练链路

该链路的设计重点是保留完整原始数据和可复现训练数据。原始数据用于审查具体样本中的三端矛盾，训练数据用于模型评估和端侧评分器生成。由于标签由规则知识库和本地大模型共同辅助生成，后续可通过人工抽样复核或规则更新逐步提高标签质量。

2.5.2 端侧评分训练链路

端侧评分链路运行在新的评分 App 中。App 启动后生成会话标识，并在本机完成 Native、WebView 和 Web 三端采集。与离线采集链路不同，端侧评分 App 加载本地探针页面，不依赖后端托管页面完成 Web 特征采集。采集完成后，端侧特征编码器按照训练阶段固化的字段顺序和编码策略构造输入向量，再调用导出的轻量评分模型得到 0 到 100 的风险分。最后，App 展示风险结果，并将风险摘要上传到后端。

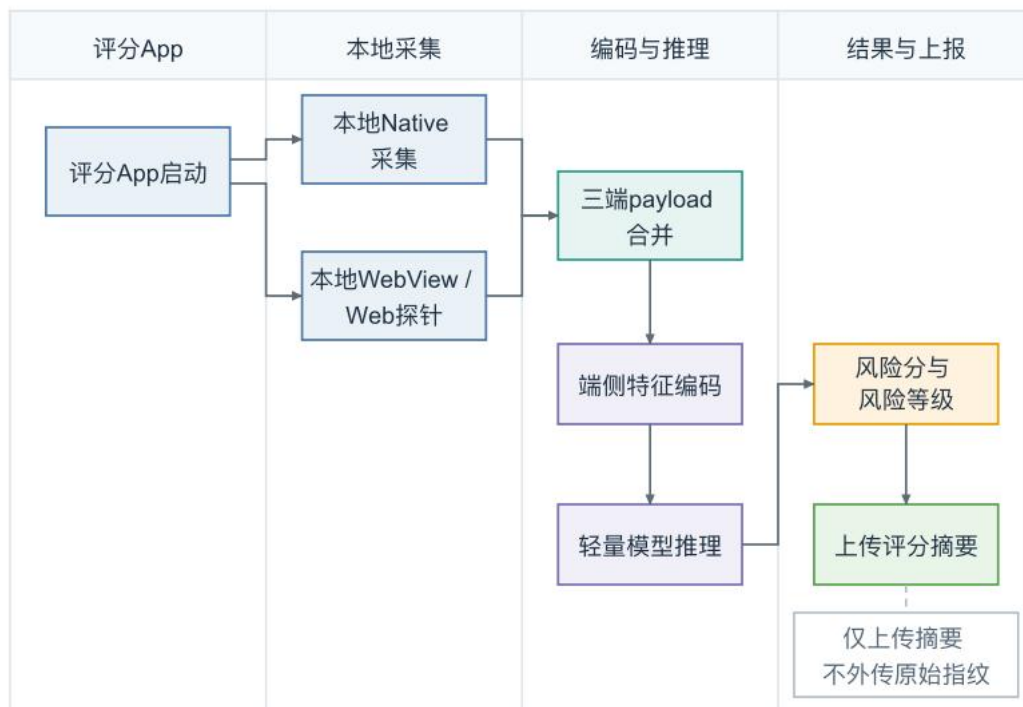


图 2.3 端侧评分运行链路

端侧链路的核心约束是训练侧与运行侧一致。若训练阶段使用 65 维特征输入，端侧编码器也必须按照相同顺序生成 `double[]` 输入；若训练阶段对类别字段和缺失值有固定处理方式，端侧也必须保持一致。否则，即使模型本身部署成功，评分结果也可能失去语义可靠性。

2.5.3 跨层一致性分析流程

跨层一致性分析位于原始采集和风险评分之间，是本文系统的核心。系统先保留 Native、WebView 和 Web 三端原始字段，再根据语义规则构造一致性特征。这些特征可以描述屏幕是否匹配、UA 是否呼应、WebView provider 是否与 Web UA 一致、JSBridge 是否存在、GPU/渲染环境是否符合移动端特征，以及传感器和调试状态是否共同指向异常环境。

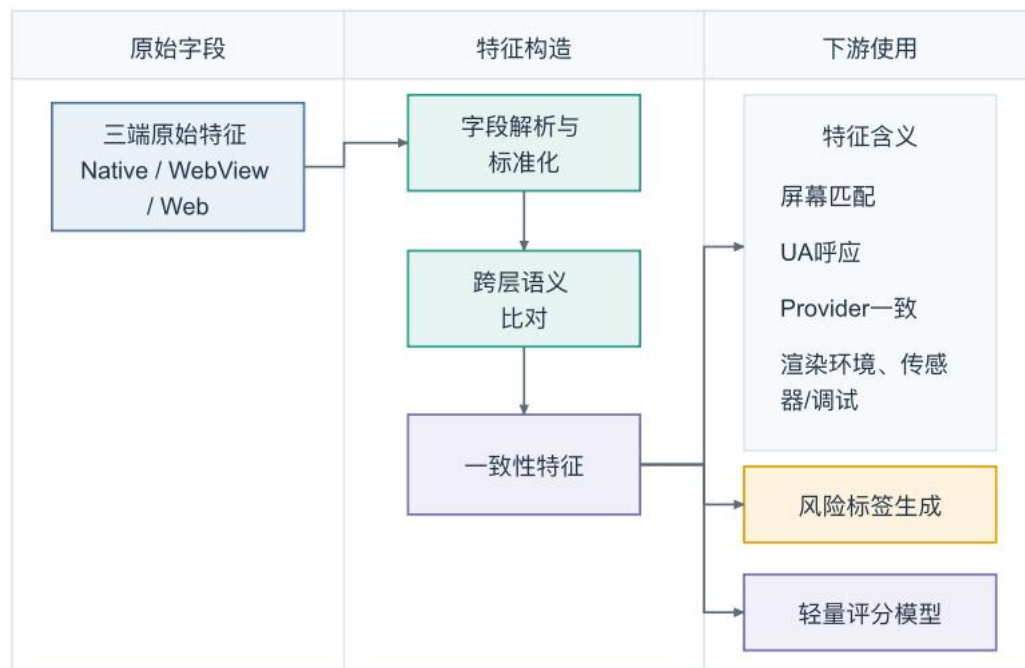


图 2.4 跨层一致性分析流程

由该流程可见，本文显式构造可解释的一致性关系，使三端字段从原始采集结果转化为跨层风险信号。一致性特征既可参与离线标签解释，也可作为轻量评分器的重要输入，从而体现跨层关系在风险识别中的作用。

2.6 本章小结

本章围绕 HybridGuard 的系统设计进行了说明。首先明确了系统的设计目标，包括三端融合一致性、无感采集、语义规则化、端侧闭环、可扩展和可复现。随后从功能需求和非功能需求两方面分析了系统需要支持的三端采集、会话合并、规则知识库、风险标签生成、端侧评分和实验分析能力。接着给出了系统总体架构，将系统划分为采集层、服务与数据层、规则与训练层和端侧评分层。最后，本章分别描述了离线标注训练链路、端侧评分运行链路和跨层一致性分析流程，为下一章关键模块的设计与实现奠定基础。



3 关键模块设计与实现

本章结合 HybridGuard 项目代码,说明三端设备指纹采集、后端会话合并、规则标注、样本生成、轻量评分器训练和 App 端侧评分闭环的实现方式。项目中保留了两条客户端链路:旧:app 模块用于三端采集并上报后端,新的:riskapp 模块用于在 App 内完成本地采集、特征编码和随机森林评分。两条链路共用同一套三端特征思想,但承担的工程角色不同。

3.1 跨端设备指纹采集模块设计与实现

3.1.1 Android Native 原生特征采集

旧采集 App 的原生特征入口位于 `android_app/HybridGuard/app/src/main/java/com/example/hybridguard/MainActivity.kt`。App 启动后生成 UUID 形式的 `sessionId`,随后在后台线程中执行 `collectAndSendNativeData()`,避免网络请求阻塞主线程。该函数将 Android 原生特征组织为 `android_native_data`,并通过 OkHttp 以 JSON 形式 POST 到后端 `/api/collect/fingerprint` 接口。

Native 侧特征采用分层 JSON 结构,主要包括六类,如表 3.1 所示。

表 3.1 Android Native 原生特征分层与实现依据

子层级	主要字段	实现依据
构建指纹层	device_model、device_brand、os_version、os_api_level、cpu_abi、build_fingerprint、build_tags、uptime_ms	Build 与 SystemClock
内存层	total_memory_gb、avail_memory_gb、is_low_memory	ActivityManager.MemoryInfo
物理屏幕层	screen_resolution_physical、screen_density_dpi、screen_xdpi、screen_ydpi、screen_scaled_density	resources.displayMetrics
电池动态层	battery_level_pct、battery_temp_celsius、battery_voltage_mv、is_charging	ACTION_BATTERY_CHANGED
传感器矩阵层	sensor_total_count、陀螺仪、加速度计、磁力计、光线、距离、气压传感器存在性	SensorManager
安全配置层	is_adb_enabled	Settings.Global.ADB_ENABLED

这种分层结构的作用是让后端能够保留原始语义边界:构建指纹用于描述设备身份,屏幕和电池用于描述物理环境,传感器矩阵用于辅助识别模拟器或低质量虚拟环境,



安全配置用于观察调试状态。后续训练和端侧评分时,系统再将这些嵌套字段展平为模型输入。

3.1.2 WebView 宿主容器特征采集

WebView 宿主特征由 WebAppInterface.kt 提供。MainActivity 在 WebView 中开启 JavaScript, 并通过 addJavascriptInterface(WebAppInterface(this, sessionId), "AndroidBridge")注入桥接对象。前端页面可以调用 AndroidBridge.getSessionId()获取与 Native 层一致的会话标识, 也可以调用 AndroidBridge.getWebViewSecurityFeatures()获取宿主容器特征。由于 JSBridge 是 Web 与 Native 之间的安全边界, Android 官方文档也提示 native bridge 需要谨慎使用^[24], 本文在系统中同时将其作为能力通道和宿主真实性信号。

getWebViewSecurityFeatures()返回的内容包括宿主调试状态、包名、安装来源、WebView provider 包名与版本、默认原生 User-Agent、明文流量配置、首次安装时间、最后更新时间、target SDK、min SDK 和系统 HTTP agent。前端探针收到该 JSON 后, 将其整理为 webview_data 的四个子层: 通信桥接层、内核容器层、宿主安全层、时间与编译层。若桥接调用异常, 还会记录异常信息。

该模块的关键意义在于补齐 Native 与 Web 之间的中间层。正常 Hybrid App 中, Web 页面应能通过 JSBridge 获得宿主信息, 并且 WebView provider、系统 UA 和 Web 前端 UA 应存在版本呼应。如果页面缺少桥接对象, 或宿主信息与 Web 层暴露信息明显不一致, 就可能说明请求并非来自预期 App 容器。

3.1.3 Web 前端运行环境特征采集

Web 前端探针位于 backend_server/index.html。旧采集链路中, Android App 加载后端托管页面, 页面在 window.onload 后执行 collectAndSend()。探针首先尝试与 AndroidBridge 建立连接, 记录 jsbridge_injected 和 bridge_latency_ms; 若桥接不存在, 则生成以 fallback-web-为前缀的临时会话标识, 并将桥接缺失作为高风险线索保留。

Web 层采集内容包括四类: 一是 Navigator 环境, 例如 user_agent、language、platform、hardware_concurrency、device_memory 和 max_touch_points; 二是屏幕显示环境, 例如逻辑分辨率、DPR、色深和可用宽高; 三是图形渲染环境, 例如 WebGL vendor、renderer、扩展数量和 Canvas 哈希; 四是执行环境, 例如 CPU 算力挑战耗时和时区偏



移。Canvas 哈希通过页面绘制固定文本和图形后计算 SHA-256 得到, 算力挑战则通过循环执行三角函数计算得到耗时。

最终, 前端将 `webview_data` 和 `web_data` 组合成 JSON, 通过 `fetch('/api/collect/fingerprint')` 上传后端。这样, 旧采集链路中 Native 数据和 Web/WebView 数据虽然分两次异步上报, 但都使用同一 `session_id` 进行会话合并。

3.1.4 端侧评分 App 中的本地采集

新 `riskapp` 模块将采集和评分闭环迁移到 App 本地。其入口为 `android_app/HybridGuard/riskapp/src/main/java/com/example/hybridguard/riskapp/MainActivity.kt`。App 启动后生成 `sessionId`, 调用 `FingerprintCollector.collectNativeFlat()` 直接采集展平后的 Native 特征, 然后加载本地资源 `file:///android_asset/local_probe.html`, 不再依赖后端托管探针页面。

本地 Web 探针通过 `ScoringWebBridge` 与 Android 端通信。该桥接对象提供四个能力: `getSessionId()` 返回会话标识, `getWebViewSecurityFeatures()` 返回 WebView 宿主特征, `sha256()` 为 Canvas 数据提供端侧哈希能力, `submitWebPayload()` 将 Web/WebView payload 回传给 Kotlin 层。`local_probe.html` 的采集字段与旧探针基本一致, 但数据不再直接上传后端, 而是交给 Android 本地评分流程。

3.2 后端数据接收、合并与持久化模块

后端服务位于 `backend_server/main.py`, 使用 FastAPI 实现。服务提供三个主要入口: 根路径/用于托管前端探针页面, `/api/collect/fingerprint` 用于接收三端原始指纹数据, `/api/risk/local-score` 用于接收新评分 App 上传的端侧评分摘要。此外, `/health` 用于健康检查。后端开启了 CORS, 便于本地测试和 WebView 页面上报。

3.2.1 Pydantic 数据模型

后端使用 Pydantic 模型定义三端数据结构。`AndroidNativeData` 对应 Native 六个子层, `WebViewData` 对应桥接、内核、宿主安全、时间编译和异常层, `WebData` 对应 Navigator、屏幕、图形和执行层。总载荷 `FingerprintPayload` 包含 `session_id`、`timestamp`、`client_ip` 以及三个可选数据块。由于 Native 与 Web/WebView 分次上报, 三个数据块均允许为空, 由后端负责合并。



端侧评分摘要使用单独的 LocalRiskScorePayload。该模型只包含 session_id、timestamp、risk_score、risk_level、risk_reason、scoring_engine 和 feature_count，不接收三端原始指纹数据。这一点与端侧数据最小化目标一致：评分 App 本地完成推理后，只把结果摘要传回服务端。

3.2.2 会话合并策略

后端使用内存字典 sessions_db 暂存会话，并在启动时尝试从 merged_sessions.json 恢复历史数据。当/api/collect/fingerprint 收到请求时，后端先检查 session_id 是否已存在；若不存在，则初始化一条包含 Native、WebView、Web 三个空槽位的记录；若已存在，则认为这是同一会话的补充数据。

合并时，代码使用 payload.dict(exclude_unset=True, exclude_none=True)提取本次请求实际携带的非空字段，只覆盖对应数据块，不会用空值擦除已有内容。该策略支持 Native 先上报、Web 后上报，或者 Web 先到达、Native 后到达的异步情况。合并后的完整嵌套会话会写回 merged_sessions.json，便于后续回溯。

3.2.3 训练数据展开

为了便于大模型标注和机器学习训练，后端在会话具备 Native 和 Web 数据后，会生成一份扁平化版本追加到 collected_data.jsonl。代码分别遍历 android_native_data、webview_data 和 web_data 的子层，将内部字段合并到单层字典中。由此，训练脚本可使用 pandas.json_normalize 展开数据，而不必在每次训练时重复处理多层嵌套结构。

需要注意的是，merged_sessions.json 保留的是原始嵌套结构，适合审计和解释；collected_data.jsonl 保存的是面向标注和训练的扁平结构，适合批处理。两者服务于不同阶段，避免了“只保留训练表而丢失原始语义”的问题。

3.3 跨层语义规则知识库与风险标签生成模块

3.3.1 规则知识库构建思路

HybridGuard 将单字段阈值、跨层语义规则和模型拟合结果结合起来，并将规则知识库设计为系统中的独立语义层，用于系统化表达“什么样的设备环境是可信的”“什么样的跨层矛盾值得警惕”“哪些异常组合更可能对应模拟器、云机房、无头浏览器或接口



重放”等风控判断。规则知识库既服务于离线风险标签生成，也服务于后续一致性特征构造和端侧轻量评分器训练，是本文区别于普通字段采集系统的重要组成部分。

规则知识库的内容可以分为三类。第一类是跨层语义对齐规则，关注 Native、WebView 和 Web 三端之间是否相互呼应。例如 Native 物理屏幕与 Web 逻辑屏幕和 DPR 是否近似对应，Native 设备型号和系统版本是否能在 Web UA 中得到印证，WebView provider 版本是否与 Web UA 中的 Chrome/WebView 主版本一致，JSBridge 返回的会话标识是否与 Native 层保持一致。这类规则用于发现“同一会话中不同层级呈现出设备来源或宿主来源不一致”的问题。

第二类是风险场景判定规则，关注单端异常和多字段组合异常。例如传感器数量极少、关键传感器缺失、电池温度长期为固定值、WebGL renderer 出现 SwiftShader 或 Headless 相关特征、platform 暴露 PC 环境、user_agent 呈现为 python-requests、安装来源为 manual 且伴随 ADB 开启或时区异常、满电且长期接电等。这类规则不一定都表现为三端字段互相矛盾，但它们能够表达移动端风控中常见的攻击和测试机房特征。

第三类是容错规则，约束大模型不要把开发阶段和真实移动端差异误判为攻击。例如 WebView 可用高度会受到状态栏、导航栏和安全区影响，Web deviceMemory 本身是粗粒度近似值，Chrome/WebView 小版本可能因系统更新存在差异，开发灰度阶段的 ADB、debuggable 和 cleartext 配置也不应单独构成高危。容错规则的作用是让知识库既能识别攻击特征，又能减少对真实设备和开发样本的误伤。

规则知识库采用大模型辅助归纳与人工筛选结合的方式构建：先向大模型输入三端字段字典、典型正常样本、典型攻击样本、攻击样本生成模板以及人工确认的风险解释，再由大模型归纳候选规则^[25]；随后对候选规则进行去重、合并、阈值修正和优先级整理，形成可解释的规则条目。工程上，本文将规则知识库同时保存为 scoring/rule_knowledge_base.md 和 scoring/rule_knowledge_base.json。前者用于论文说明和人工审阅，后者用于批量评分脚本读取。每条结构化规则包含规则编号、规则类别、关联字段、触发条件、风险等级、建议分数区间、是否一票否决、容错说明和解释模板，从而使规则库具备扩展、审计和版本管理能力^[26]。

3.3.2 大模型辅助风险评分流程

大模型辅助评分流程以规则知识库为约束，对三端指纹样本进行可解释的风险判



断。流程上,系统先将采集到的 Native、WebView 和 Web 数据整理为统一 JSON,再将规则知识库中的核心规则、优先级和容错说明提供给大模型,要求模型按照规则逐项分析样本,最终输出 risk_score 和 risk_reason。其中 risk_score 用于后续训练轻量评分器, risk_reason 用于保留样本的风险解释。

项目中包含两个相关脚本。backend_server/rba_engine.py 更偏交互式分析,调用本地 LM Studio 的 OpenAI-compatible API,并使用流式输出展示分析过程。它的提示词强调跨层交叉验证,包括物理生态链路、渲染与算力对齐、屏幕和 UA 撕裂、JSBridge 存活等维度,适合用来观察单条样本的推理过程。

批量标注主要由 scoring/sorting_rule_kb.py 完成。该脚本同样连接本地 LM Studio 服务,但关闭流式输出以提高批处理效率。与交互式分析脚本不同,批量标注脚本会先读取 scoring/rule_knowledge_base.json,将规则库中的评分区间、一票否决规则、跨层一致性规则、攻击场景规则和容错规则统一放入 system prompt,再把单条三端指纹样本作为 user prompt 输入大模型。模型需要先按照规则编号分析命中情况,再输出严格 JSON 格式的 risk_score 和 risk_reason。

这种做法使大模型的作用由“自由判断”转为“在规则知识库约束下进行可解释匹配”。例如传感器数量过少或 JSBridge 缺失将触发一票否决规则; manual 安装、ADB、UTC 时区和满电组合可被判定为云机房或测试机架特征;型号、系统版本、屏幕/DPR、CPU/platform、硬件/GPU、WebView provider 和 UA 主版本用于判断三端是否自洽;而屏幕高度误差、内存粗粒度差异、开发阶段调试配置和 Chrome 小版本差异则被纳入容错说明。最终得到的离线风险分既保留了大模型的语义归纳能力,又受到结构化规则库约束,便于后续用轻量模型学习和压缩。

3.3.3 标签质量控制

批量标注脚本实现了若干工程容错机制。第一,输出文件存在时先读取已处理过的 session_id,实现断点续传,避免批处理中断后重复标注。第二,模型输出可能包含草稿、思考内容或格式噪声,脚本会优先提取包含 risk_score 和 risk_reason 的 JSON 对象,并对部分中文符号和转义字符做修正。第三,标注结果并不覆盖原始样本,而是追加到 llm_label 字段中,保留原始三端指纹和风险标签的对应关系。

大模型主要服务于离线风险标签生成,App 运行时由端侧轻量评分器完成实时风险



输出。这样既能利用大模型对跨层语义关系的解释能力,也避免端侧实时风控依赖大模型服务带来的成本、延迟和稳定性问题^[34]。

3.3.4 规则示例

表 3.2 给出本文系统中具有代表性的规则类型。表中既包括跨层一致性规则,也包括风险场景判定规则和容错规则。

表 3.2 跨层语义规则类型与风险解释

规则类别	关联特征层	规则含义	风险解释
屏幕一致性	Native 屏幕 + Web 屏幕	逻辑分辨率 x DPR 应近似对应物理分辨率	偏差过大可能说明 Web 环境被伪造或脱离宿主
UA 一致性	Native Build + WebView UA + Web UA	系统版本、机型和内核版本应相互呼应	不一致可能说明 UA 被改写或请求被重放
宿主真实性	JSBridge + session_id	Web 页面应能通过 JSBridge 获取同一会话	JSBridge 缺失可能说明外部浏览器或脚本绕过 App
物理可信性	传感器 + 电池	真机通常具备合理传感器矩阵和动态电池状态	传感器极少或电池死值可能指向模拟器
渲染环境	WebGL + Native 硬件	GPU renderer 应符合移动端硬件生态	SwiftShader、Headless、P C GPU 属于高风险线索
云机房/调试环境	安装来源 + ADB + 电池 + 时区	manual 安装、ADB 开启、满电接线、异常时区组合出现时提高风险	可能对应云真机、测试机架或群控设备
自动化/重放	Native 缺失 + JSBridge 缺失 + UA	Native 字段为空、桥接缺失、UA 为脚本客户端时判为高危	可能是接口重放或脱离 App 宿主的自动化请求
容错规则	开发配置 + 设备差异	ADB/debuggable、屏幕高度误差、内存波动不应单独高危	避免把开发阶段或真实设备差异误判为攻击

3.4 数据集构建与样本生成模块

3.4.1 真实设备数据采集

真实样本主要来自旧采集 App 在 Android 真机和云真机环境中的运行结果。旧 app 模块加载后端探针页, Native 与 Web/WebView 分别上报后由后端合并。仓库中 scoring/real_data.jsonl 当前包含 84 条真实采集样本。项目还包含 run_browserstack.py, 用于通过 Appium 和 BrowserStack 云真机批量启动 App 并等待探针静默上报。

3.4.2 真实样本扩充

scoring/augment_device_data.py 用于对真实样本进行扩充, 默认目标数量为 300 条。



脚本保留设备型号、系统版本、屏幕、传感器等相对静态特征，同时扰动时间戳、session_id、开机时间、可用内存、电池电量、电池温度、充电状态、JSBridge 延迟和算力挑战耗时等动态特征。这样可以模拟同一真实设备在不同时间运行时产生的自然波动，降低模型对少量固定样本的记忆倾向。

3.4.3 高危样本生成

scoring/generate_bad_data.py 用于生成 300 条高危模拟样本，覆盖三类攻击模板。第一类是 API 重放，将 Native 字段置空，关闭 JSBridge，并使用 python-requests 形式的 UA，模拟脱离 App 宿主的接口请求。第二类是无头 PC 浏览器，将设备模型伪造成 Windows PC，传感器置为缺失，平台设置为 Win32，UA 使用 HeadlessChrome，并将算力耗时设置得异常偏快。第三类是廉价模拟器，使用 goldfish、ranchu、x86、Google SwiftShader、UTC 时区和极少传感器等典型模拟器特征。

3.4.4 当前数据规模

当前仓库中的主要数据文件如表 3.3 所示。

表 3.3 主要数据文件规模与用途

文件	当前行数	用途
scoring/real_data.jsonl	84	真实采集样本
scoring/real_data_augmented.jsonl	300	真实样本扩充结果
scoring/simulated_bad_data.jsonl	300	高危模拟样本
backend_server/collected_data.jsonl	808	后端采集链路追加的训练候选数据
training/scored_data.jsonl	1323	带 llm_label 的训练与实验数据
backend_server/local_score_results.jsonl	2	端侧评分 App 上传的评分摘要样例

3.5 轻量风险评分模块

3.5.1 特征工程

轻量评分器训练脚本位于 training/目录。train_randomforest.py 读取 scored_data.jsonl，使用 pandas.json_normalize 展开嵌套 JSON，并以 llm_label.risk_score 作为回归目标。训练输入会剔除 session_id、timestamp、client_ip、llm_label.risk_score 和 llm_label.risk_reason 等非输入字段。对字符串和布尔字段，脚本使用 LabelEncoder 编码，并将缺失值填为"Unknown"；对数值字段，缺失值填为-1。

神经网络对照脚本 train_mlp.py 采用另一套预处理方式：删除 build_fingerprint、us



er_agent 和 canvas_hash 等高维复杂文本字段, 对类别特征做独热编码, 对数值特征用中位数填补并使用 StandardScaler 标准化。两种脚本使用相同的 train_test_split(test_size=0.2, random_state=42), 便于在相近划分下比较工程效果。

3.5.2 候选评分器实现

随机森林脚本使用 RandomForestRegressor(n_estimators=50, max_depth=10, random_state=42)。限制树深度的原因是控制生成 Java 代码的体积, 避免端侧编译和运行开销过大。训练完成后, 脚本使用 m2cgen 将模型导出为 DeviceRiskScorer.java, 并生成 tree_test_predictions_results.csv 供误差分析。

MLP 脚本定义了一个浅层网络 ShallowRiskNet, 结构为 input -> 64 -> 32 -> 1, 中间使用 ReLU 和 Dropout, 损失函数为 MSE, 优化器为 Adam, 训练 150 个 epoch。训练结束后保存 shallow_risk_net.pth 和 feature_scaler.pkl。该模型作为随机森林的工程对照, 用于评估结构化三端指纹在轻量模型中的拟合效果。

3.5.3 端侧部署模型选择依据

本文最终选择随机森林作为端侧评分器, 主要基于工程部署约束。三端设备指纹属于小样本、结构化、混合数值与类别特征的数据, 随机森林对这类数据较稳定, 训练和部署成本较低; m2cgen 可以直接生成 Java 推理代码, 便于集成到 Android 工程; 同时, 树模型特征重要性和分裂路径也更容易用于解释风险判断。端侧模型需要兼顾推理速度、资源占用和可维护性, 因此轻量化部署比复杂模型结构更符合本文系统目标^{[9][33]}。

3.6 App 本地评分模块设计与实现

3.6.1 端侧模块结构

新评分 App 位于 android_app/HybridGuard/riskapp/。其 build.gradle.kts 中设置 minSdk=30、targetSdk=36, 依赖 AppCompatActivity、Material 和 OkHttp。该模块的核心文件包括 MainActivity.kt、FingerprintCollector.kt、ScoringWebBridge.kt、RiskFeatureEncoder.java、DeviceRiskScorer.java 和 assets/local_probe.html。Android 官方文档对 WebView 本地内容加载、资源组织和安全加载方式给出了专门说明, 这为端侧本地探针页面的工程实现提供了参考^[35]。端侧评分流程如下。

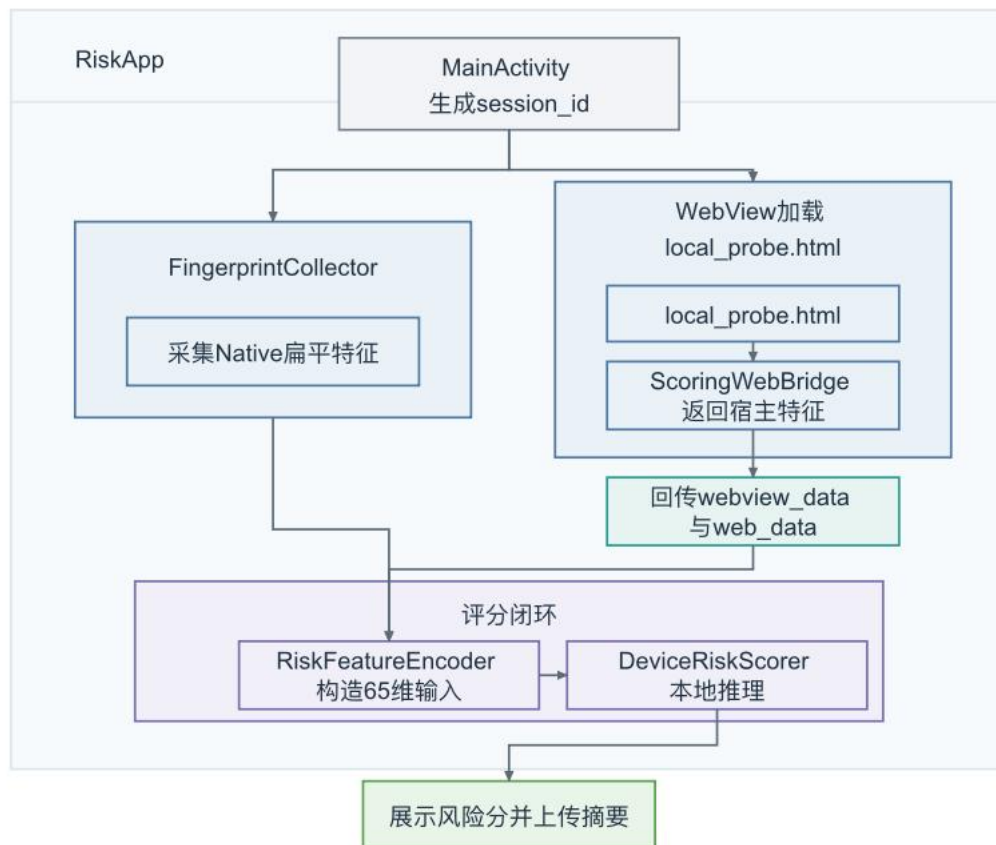


图 3.1 端侧模块结构：调度、采集、桥接与评分

3.6.2 端侧特征编码

RiskFeatureEncoder.java 是端侧特征适配器，固化了训练阶段 `pandas.json_normalize` 后的 65 维字段顺序。字段覆盖 Native 33 维、WebView 14 维和 Web 18 维。编码器内部维护每个类别字段的取值列表，布尔值转为"True"或"False"后参与类别编码，缺失类别优先映射为"Unknown"，无法识别时返回-1.0；数值字段直接转为 double，缺失或解析失败时返回-1.0。

这一层是端侧评分闭环中最关键的工程约束。训练阶段的列顺序、类别编码和缺失值策略必须与 Android 端完全一致，否则 DeviceRiskScorer 得到的输入语义会错位。项目通过将 65 维字段顺序和类别表固化进 Java 类，避免运行时依赖 Python 预处理环境。

3.6.3 本地随机森林推理与结果上报

MainActivity.kt 在收到 local_probe.html 通过 submitWebPayload()回传的 Web/WebView 数据后，先使用 FingerprintCollector.flattenLayers()将嵌套数据展平，再构造包含 session_id、timestamp、android_native_data、webview_data 和 web_data 的评分会话。随



后调用 `RiskFeatureEncoder.encode(scoringSession)` 生成输入向量, 并调用 `DeviceRiskScorer.score(features)` 得到风险分。输出分数被限制在 0 到 100 之间。

风险等级采用简单阈值: 分数大于等于 80 为 high, 大于等于 50 为 medium, 其余为 low。App 会在界面上展示本地评分结果, 并向 `/api/risk/local-score` 上传评分摘要。上传字段包括 `session_id`、`timestamp`、`risk_score`、`risk_level`、`risk_reason`、`scoring_engine=random_forest_m2cgen` 和 `feature_count`。该接口不上传原始三端指纹, 体现了端侧闭环和数据最小化设计。

3.6.4 工程风险与处理

端侧评分实现中最主要的风险是训练侧和端侧特征不一致。项目通过 `RiskFeatureEncoder` 固化字段顺序、类别编码和缺失值策略来降低该风险。第二个风险是 `WebView` 采集异步完成前就触发评分。当前实现由 `local_probe.html` 采集完成后主动调用 `submitWebPayload()`, Android 端收到完整 `payload` 后才执行评分。第三个风险是端侧生成的 Java 模型代码较大, 因此训练脚本限制随机森林深度, 以控制 Android 编译和运行成本。

3.7 本章小结

本章基于项目实际代码说明了 `HybridGuard` 的关键模块实现。旧 app、后端服务、`scoring` 训练数据链路、`training` 模型训练链路和新 `riskapp` 端侧评分链路共同构成从三端采集、会话合并、规则标注到本地评分的工程闭环。其中, 后端保留原始嵌套会话与训练用 JSONL 数据, 离线侧将规则知识库转化为风险标签, 端侧则通过固定 65 维特征编码和随机森林 Java 推理代码复现离线评分能力。由此, 系统从原始指纹采集推进到可部署的本地风险分输出。



4 实验设计与结果分析

本章基于 HybridGuard 已采集、构造和标注的三端设备指纹数据,对系统有效性进行实验分析。实验重点考察三端原始特征的信息量、跨层一致性特征对规则知识库风险判断的表达能力,以及相关语义特征在分组验证下的泛化表现。

4.1 实验环境与数据集说明

4.1.1 实验环境

本章实验依托 HybridGuard 原型系统完成。客户端侧包括 Android 原生采集模块、WebView 宿主采集模块和 Web 前端探针模块;后端侧使用 FastAPI 接收三端数据,按 session_id 完成会话合并,并将合并结果保存为 JSON 与 JSONL 文件;离线侧使用综合规则知识库驱动大模型生成风险标签,再使用轻量模型进行风险分拟合和消融评估。

风险标签生成阶段使用第三章介绍的综合规则知识库。该知识库整合跨层一致性、核心完整性、物理与运行环境、攻击场景聚合和容错规则。大模型在标注时接收三端设备指纹 JSON 与规则知识库,输出结构化的 risk_score 和 risk_reason,因此本章风险分是基于规则知识库进行综合分析后得到的连续评分。

轻量评估器采用随机森林回归模型,参数与训练侧和消融实验脚本保持一致:

```
RandomForestRegressor(n_estimators=50, max_depth=10, random_state=42)
```

随机森林在本章中用于评估不同特征组合对规则知识库风险标签的拟合能力,并作为端侧轻量评分器的工程实现。所有消融实验脚本、指标表和图表均保存在 ablation/目录下。

4.1.2 数据来源与样本规模

本章主要实验数据来自 training/scored_data.jsonl,共 1323 条带风险标签样本。每条样本包含 Android Native、WebView 宿主和 Web 前端三类特征,并包含规则知识库驱动的大模型离线标注结果 llm_label.risk_score。分组脚本根据设备字段、安装来源、调试状态、时区、UA 和攻击模板等启发式信息,将样本划分为真实物理设备、云真机/测试环境和脚本攻击三类来源。



表 4.1 实验样本来源类型与分组规模

来源类型	样本数	分组数	含义
physical_device	286	65	未命中云测和脚本攻击启发式的真实设备类样本
cloud_device	737	77	安装来源、时区、ADB 或云测特征命中测试环境启发式样本
script_attack	300	3	API 重放、无头 PC 浏览器、廉价模拟器等脚本攻击模板

需要区分“样本数”和“分组数”。真实样本扩充、云真机采集和脚本攻击模板都可能产生相似样本。如果只按行随机切分，同一设备或同一攻击模板的近似记录可能同时出现在训练集和测试集中，使指标偏乐观。因此，本章除随机 holdout 外，还采用按 group_id 切分的分组交叉验证。数据来源构成图见 ablation/figures/figure_01_source_distribution.png。

4.1.3 风险标签与分数分布

实验样本中的风险分范围为 0 到 100。根据 training/scored_data.jsonl 中的标签统计，主要分布如表 4.2 所示。

表 4.2 风险标签分数分布

风险分数	样本数	说明
15	297	低风险正常设备或较可信环境
20	1	低风险样本
35	10	较低风险但存在轻微异常
38	57	低到中风险边界样本
40	55	中低风险样本
42	348	云真机或测试环境常见中风险样本
45	255	云测、调试或轻微异常环境
92	63	高风险攻击样本
95	159	高风险攻击样本
98	78	高风险攻击样本

本章将 risk_score >= 80 作为高风险二分类阈值，对应 300 条高风险样本。连续风险分用于计算 MAE、RMSE 和 R2，高风险阈值用于计算 Precision、Recall、F1 和 Accuracy 等辅助分类指标。由于标签来自规则知识库驱动的大模型离线标注流程，标签具有规则解释性，但仍需要在后续工作中结合人工复核和真实业务样本继续校准。

4.1.4 三端采集字段完整性统计

除已标注训练数据外，backend_server/collected_data.jsonl 中保存了后端采集链路追



加的 808 条有效记录。在这些记录中, 样本均包含 Native、WebView 和 Web 三端字段, 满足后续消融分析对字段完整性的要求。

表 4.3 三端采集特征层字段规模

特征层	字段数	代表字段
Android Native	33	设备型号、品牌、系统版本、CPU ABI、物理屏幕、电池、传感器、ADB 状态
WebView 宿主	14	JSBridge 注入、桥接延迟、WebView Provider、系统 UA、安装来源、debuggable
Web 前端	18	Navigator、逻辑屏幕、DPR、WebGL、Canvas、算力耗时、时区

三端字段合计为 65 维原始特征。后续实验中的 Raw all 配置即使用这 65 维原始三端特征作为完整基线。字段完整性统计用于说明实验数据具备三端对齐基础, 为后续消融实验提供数据前提。

4.2 实验目标、实验设计与评估指标

4.2.1 实验目标

本章实验围绕三个问题展开: 第一, 三端原始特征分别包含多少风险信息; 第二, 跨层一致性关系是否比直接拼接原始字段更能体现系统价值; 第三, 在避免同源设备或同源攻击模板泄漏后, 核心一致性特征是否仍具有泛化能力。这三个问题分别对应三端采集、规则知识库和端侧轻量评分闭环的有效性验证。

4.2.2 评估指标

由于风险标签是 0 到 100 的连续分数, 本章以回归指标作为主要评估指标, 包括平均绝对误差 MAE、均方根误差 RMSE 和决定系数 R2。

$$\text{MAE} = \text{mean}(|y_{\text{true}} - y_{\text{pred}}|)$$

$$\text{RMSE} = \sqrt{\text{mean}((y_{\text{true}} - y_{\text{pred}})^2)}$$

$$\text{R2} = 1 - \text{SSE} / \text{SST}$$

MAE 反映预测风险分与标签风险分之间的平均偏差, RMSE 对较大误差更敏感, R2 表示模型对风险分方差的解释程度。与此同时, 本章将 $\text{risk_score} \geq 80$ 作为高风险阈值计算 Precision、Recall、F1 和 Accuracy。高风险 F1 仅作为辅助指标, 因为当前高风险样本主要来自规则明确的攻击模板, 随机划分下容易出现饱和。

4.2.3 数据划分方式



本章使用两种数据划分方式。第一种是随机 holdout，固定 80/20 划分，训练集 1058 条，测试集 265 条，随机种子为 42。该划分用于快速比较不同特征组的信息量。

第二种是分组交叉验证。该方法按 group_id 切分数据，同一真实设备、同一云测设备或同一脚本攻击模板不能同时出现在训练集和测试集中。由于脚本攻击样本由 3 个主模板生成，因此采用 3 折分组交叉验证，使每一折测试集都包含一个未见脚本攻击模板，同时包含一定数量的真实设备组和云测设备组。

表 4.4 分组交叉验证各折样本分布

Fold	physical_device	cloud_device	script_attack
1	82 rows / 21 groups	290 rows / 23 groups	104 rows / 1 group
2	128 rows / 21 groups	236 rows / 28 groups	97 rows / 1 group
3	76 rows / 23 groups	211 rows / 26 groups	99 rows / 1 group

分组划分比随机 holdout 更严格，更接近真实部署时遇到新设备和新攻击变体的情况。Fold 构成图见 ablation/figures/figure_02_fold_distribution.png，holdout 与分组 MAE 对比图见 ablation/figures/figure_03_holdout_vs_grouped_mae.png。

4.3 粗粒度三端特征消融实验

4.3.1 实验目的与分组

粗粒度三端消融实验按特征来源划分 Native、WebView 和 Web 三类原始字段，比较单端、双端和完整三端组合对风险分的拟合效果。

表 4.5 粗粒度三端特征消融实验配置

配置	使用特征	特征数	实验目的
Native only	Android Native	33	验证底层硬件、系统、电池、传感器等信息量
WebView only	WebView 宿主	14	验证 JSBridge、安装来源、Provider、宿主安全信息量
Web only	Web 前端	18	验证 UA、WebGL、Canvas、DPR、算力等信息量
Native + WebView	Native + WebView	47	验证底层系统与宿主容器组合
Native + Web	Native + Web	51	验证底层系统与前端运行环境组合
WebView + Web	WebView + Web	32	验证宿主容器与前端环境组合
Native + WebView + Web	三端完整原始特征	65	完整三端基线

该实验属于“按端删除”的直观消融，适合观察每一端原始字段的信息量。但三端原始字段之间存在天然冗余，例如设备型号、系统版本、屏幕、WebView provider 和



UA 会在不同层级中相互体现, 因此本节结果还需要结合后续一致性特征实验解释。

4.3.2 随机 Holdout 结果与分析

随机 holdout 下的粗粒度消融结果如表 4.6 所示。

表 4.6 粗粒度三端特征随机 Holdout 结果

配置	特征数	MAE	RMSE	R2	高风险 F1
Native only	33	1.222	2.198	0.9933	1.000
WebView only	14	1.139	1.537	0.9967	1.000
Web only	18	1.416	2.377	0.9922	1.000
Native + WebView	47	1.129	1.515	0.9968	1.000
Native + Web	51	1.219	2.177	0.9934	1.000
WebView + Web	32	1.131	1.537	0.9967	1.000
Native + WebView + Web	65	1.140	1.544	0.9967	1.000

所有配置在随机 holdout 下都取得较低误差, 高风险 F1 均为 1.000。其中 WebView only 的 MAE 为 1.139, 接近完整三端原始特征; Native + WebView 的 MAE 为 1.129, 是该表中最低值; 完整三端原始特征的 MAE 为 1.140, 与最优配置保持接近。

这一结果说明当前数据中的风险标签包含较强规则信号, 且三端原始字段之间存在代理关系。WebView 层的 JSBridge、安装来源、debuggable、Provider 和 system UA 与规则标签高度相关; Native 层的设备、系统和屏幕信息也会部分反映到 Web UA、DPR 和 WebGL。随机划分下, 同源样本可能同时出现在训练集和测试集中, 因此粗粒度消融更适合说明各端字段的信息量, 三端融合价值还需要通过跨层一致性特征进一步验证。

4.3.3 分组交叉验证结果与分析

在分组交叉验证下, 粗粒度消融结果更加保守。

表 4.7 粗粒度三端特征分组交叉验证结果

配置	特征数	MAE	RMSE	R2	高风险 F1
Native only	33	2.799	1.541	4.774	0.667
WebView only	14	1.541	0.435	2.643	1.000
Web only	18	12.188	7.175	23.381	0.333
Native + WebView	47	2.202	0.840	3.767	1.000
Native + Web	51	6.573	3.757	12.522	0.333
WebView + Web	32	3.053	0.994	5.100	0.978
Native + WebView + Web	65	2.642	1.004	4.455	1.000

与随机 holdout 相比, 分组验证下整体误差明显上升, 尤其是 Web only 的 MAE



达到 12.188, 说明单独 Web 前端特征对未见设备或未见模板的泛化不足。WebView only 和 Native + WebView 表现较强, 主要因为 JSBridge 注入、安装来源、debuggable、WebView provider、system HTTP agent 等字段直接体现宿主真实性和测试环境特征。

完整三端原始字段在分组验证中并未取得最优结果, 说明 Web 层原始字段在未见设备或未见模板上可能引入噪声。该结果进一步引出后续实验: 将三端之间可解释的对齐、互证和矛盾关系显式编码, 以减少原始字段堆叠带来的噪声。

4.4 跨层一致性特征构造与消融实验

4.4.1 实验目的

粗粒度三端消融用于回答“哪一端字段有信息量”, 而 Native、WebView 和 Web 三端之间是否形成可互证的语义关系, 还需要通过跨层关系进一步分析。三端融合的价值体现在不同层级之间的相互校验、相互印证和矛盾发现能力。因此, 本节将三端之间的语义对齐关系编码为显式一致性特征, 再评估这些特征对风险分拟合的作用。

4.4.2 一致性特征构造方法

一致性特征来源于综合规则知识库中的跨层语义规则, 并由 ablation/run_consistency_ablation.py 实现为数值特征。本实验共构造 38 个 consistency_* 特征, 分为四组, 如表 4.8 所示。

表 4.8 跨层一致性特征分组与语义

分组	特征数量	语义
Native-Web	18	Android 底层环境与 Web JS 暴露环境是否一致
Native-WebView	8	Android 底层环境与 App/WebView 宿主是否一致
WebView-Web	5	WebView 容器与 Web 前端运行环境是否一致
Tri-layer semantic	7	三端共同参与的核心风险规则与综合一致性分

其中的代表性特征如表 4.9 所示。

这些特征采用宽松匹配策略。例如, 屏幕一致性需要将 Web 逻辑像素乘以 DPR 后与 Native 物理像素比较, 并允许状态栏、导航栏和安全区造成的误差; GPU 关系也需要基于硬件族建立近似映射。Tri-layer semantic 组则综合三端信息判断核心完整性、传感器与 JSBridge 是否失败、manual 安装来源是否伴随时区或 ADB、官方安装来源与核心完整性是否同时通过, 以及多个一致性检查失败的聚合数量。



表 4.9 代表性一致性特征及含义

特征	含义
consistency_native_web_model_ua_strength	Native 设备型号、产品、主板、品牌在 Web UA 中的宽松匹配强度
consistency_native_web_android_version_delta	Native Android 版本与 Web UA Android 版本差异
consistency_native_web_screen_score	物理屏幕、Web 逻辑屏幕、DPR 的软一致性分
consistency_native_web_gpu_family_score	Native 硬件族与 WebGL GPU 族的宽松匹配分
consistency_native_webview_agent_model_match	Native 设备型号是否出现在 WebView system HTTP agent
consistency_webview_web_chrome_major_match	WebView Provider 与 Web UA Chrome 主版本是否一致
consistency_tri_layer_sensor_bridge_fail	传感器严重缺失或 JSBridge 未注入
consistency_tri_layer_failure_count	多个一致性检查失败的聚合计数

4.4.3 一致性消融实验分组

一致性消融实验分组设置如表 4.10 所示。

表 4.10 跨层一致性消融实验配置

特征	含义
Raw all	三端原始 65 维特征
Raw cleaned	删除高基数字符串和直接身份字段后的原始特征
Consistency only	只使用 38 个跨层一致性特征
Raw all + Consistency	原始三端特征 + 一致性特征
Raw cleaned + Consistency	清理后原始特征 + 一致性特征
Native-Web consistency	只使用 Native-Web 一致性特征
Native-WebView consistency	只使用 Native-WebView 一致性特征
WebView-Web consistency	只使用 WebView-Web 一致性特征
Tri-layer semantic	只使用三端语义规则特征

其中 Raw cleaned 用于减少模型对高基数字符串和直接身份字段的记忆，例如 build_fingerprint、user_agent、canvas_hash、屏幕分辨率、WebView provider version 和 WebGL renderer 等。Android 安全检测中的特征子集选择研究也表明，对高维结构化特征进行筛选有助于降低冗余并提升模型可解释性^[36]。

4.4.4 随机 Holdout 结果与分析

随机 holdout 下的一致性消融结果如表 4.11 所示。



表 4.11 跨层一致性消融随机 Holdout 结果

配置	特征数	MAE	RMSE	R2	高风险 F1
Raw all	65	1.140	1.544	0.9967	1.000
Raw cleaned	40	1.137	1.520	0.9968	1.000
Consistency only	38	1.103	1.473	0.9970	1.000
Raw all + Consistency	103	1.108	1.501	0.9969	1.000
Raw cleaned + Consistency	78	1.107	1.496	0.9969	1.000
Native-Web consistency	18	1.438	2.745	0.9895	1.000
Native-WebView consistency	8	2.479	4.591	0.9708	1.000
WebView-Web consistency	5	8.079	10.514	0.8467	1.000
Tri-layer semantic	7	1.111	1.469	0.9970	1.000

Consistency only 的 MAE 为 1.103, RMSE 为 1.473, 略优于完整原始三端特征 Raw all。Tri-layer semantic 仅使用 7 个特征, 也取得 MAE 1.111 和 RMSE 1.469, 接近甚至略优于完整原始特征。这说明规则知识库中的“字段关系”能够被轻量模型有效利用, 少量高质量语义规则即可表达主要风险判断逻辑。

Raw all + Consistency 和 Raw cleaned + Consistency 没有明显优于 Consistency only, 说明当前标签体系下, 显式一致性特征已经覆盖主要风险信息。WebView-Web consistency 单独效果较弱, 表明仅考察 WebView provider 与 Web UA、JSBridge 和 WebView token 的关系不足以完整判断风险, 仍需要 Native 层参与。

4.5 分组交叉验证下的跨层一致性泛化分析

4.5.1 实验目的

随机 holdout 可能让同一设备扩充样本、同一云真机环境或同一攻击模板样本同时进入训练集和测试集, 从而高估模型泛化能力。因此, 本节采用分组交叉验证, 评估未见设备、未见云测环境和未见攻击模板上的表现。如果某组特征在分组验证中仍保持较低误差, 则说明其语义关系具有更强泛化价值。

4.5.2 分组版一致性消融结果

分组交叉验证下的一致性消融结果如表 4.12 所示。

对应图表见 ablation/figures/figure_04_grouped_main_results.png 和 ablation/grouped_consistency_error_metrics.png。与随机 holdout 相比, 分组验证下大多数配置误差上升, 说明分组评估确实更加严格。



表 4.12 分组交叉验证下的一致性消融结果

配置	特征数	MAE	RMSE	R2	高风险 F1
Raw all	65	2.642	1.004	4.455	1.000
Raw cleaned	40	2.617	1.426	4.723	0.680
Consistency only	38	3.388	1.265	5.735	0.333
Raw all + Consistency	103	3.407	1.447	5.831	0.519
Raw cleaned + Consistency	78	3.331	1.777	5.846	0.667
Native-Web consistency	18	12.733	3.042	21.783	0.333
Native-WebView consistency	8	2.608	0.353	4.408	1.000
WebView-Web consistency	5	7.792	0.563	10.486	1.000
Tri-layer semantic	7	2.281	0.466	3.358	1.000

4.5.3 结果分析

Consistency only 在随机 holdout 中略优于 Raw all，但在分组交叉验证中 MAE 上升到 3.388，不再优于 Raw all 的 2.642。这说明 38 个一致性特征中既包含强语义规则，也包含依赖设备表达方式的宽松匹配规则。当测试集包含未见设备或未见模板时，部分型号、屏幕、GPU 和 UA 关系可能出现新的表达形式，导致整体误差上升。

相比之下，Tri-layer semantic 在分组验证中表现最好，MAE 为 2.281，RMSE 为 3.358，优于完整原始三端特征 Raw all。该配置仅使用 7 个三端语义规则特征，却在更严格设置下取得最低误差，说明传感器与 JSBridge 完整性、manual 安装来源与时区或 ADB 的组合、核心链路通过情况和失败计数等规则不依赖具体设备型号字符串，更容易跨设备和跨模板泛化。

Native-WebView consistency 的 MAE 为 2.608，也优于 Raw all，说明 App 宿主真实性、JSBridge、安装来源、debuggable 和 system HTTP agent 是稳定风险信号。Native-Web consistency 的 MAE 达到 12.733，则说明单独依赖型号、屏幕、GPU 与 UA 的宽松匹配不足以处理未见设备，后续需要扩大设备覆盖并完善硬件族、GPU、UA 和屏幕规则。

综合上述结果，在更严格的分组交叉验证下，三端语义规则特征仅使用 7 个特征便取得优于完整三端原始特征的误差表现，说明跨层一致性建模在未见设备和未见攻击模板上仍具有一定泛化能力。

4.6 特征重要性与跨层规则解释

4.6.1 实验目的



前几节主要从误差指标说明一致性特征有效。本节进一步分析随机森林在 Consistency only 配置下依赖哪些特征，以验证模型是否确实利用了跨层语义关系。特征重要性结果来自 ablation/consistency_top_feature_importance.csv，对应图见 ablation/figures/figure_05_consistency_feature_importance.png。

4.6.2 重要特征结果

Consistency only 配置下重要性排名较高的特征如表 4.13 所示。

表 4.13 重要一致性特征及风险解释

特征	重要性	解释
consistency_tri_layer_sensor_bridge_fail	0.1518	传感器严重缺失或 JSBridge 未注入，对高风险样本关键
consistency_native_webview_debug_cleartext_tension	0.1017	debug 与明文网络配置共同反映测试或调试宿主特征
consistency_native_web_model_ua_strength	0.0850	Native 设备身份与 Web UA 是否能相互印证
consistency_tri_layer_failure_count	0.0844	多个一致性检查失败的聚合信号
consistency_webview_web_ua_has_wv_token	0.0843	Web UA 是否呈现 WebView 运行时特征
consistency_native_web_model_ua_match	0.0839	Native 型号是否与 Web UA 直接匹配
consistency_tri_layer_manual_timezone_or_adb	0.0749	manual 安装来源结合时区或 ADB 的云机房风险

排名第一的 consistency_tri_layer_sensor_bridge_fail 同时涉及 Native 物理可信性和 WebView 宿主真实性。传感器严重缺失说明底层设备生态不完整，JSBridge 未注入说明页面可能脱离 App 宿主，因此二者都是规则知识库中的核心完整性信号。

consistency_native_webview_debug_cleartext_tension 和 consistency_tri_layer_manual_timezone_or_adb 反映测试环境和云机房特征。单独的 debug、cleartext、manual、ADB 或时区异常未必高危，但组合出现时更接近测试机架、云真机或群控环境。consistency_native_web_model_ua_strength、consistency_native_web_model_ua_match 和 consistency_webview_web_ua_has_wv_token 则说明设备身份一致性和 WebView 宿主标记仍是重要辅助依据。

4.6.3 规则解释与系统贡献对应关系

从规则类别看，特征重要性与本文系统设计高度对应。首先是宿主真实性，JSBri



dge 注入、WebView UA 标记、WebView provider 与 Chrome 主版本一致性用于判断页面是否运行在预期 App 宿主中。其次是设备身份一致性, Native 型号、产品、品牌、Android 版本与 Web UA 的宽松匹配用于判断不同层级是否具有同一设备来源特征。第三是物理可信性, 传感器矩阵、电池状态、ADB、安装来源等特征组合用于区分真实用户设备、云真机/测试机架和模拟器。第四是渲染与运行环境一致性, WebGL renderer、GPU family、Headless、SwiftShader、platform 和算力耗时等特征用于识别桌面环境、无头浏览器和软渲染模拟器。

总体而言, 特征重要性结果表明, 模型主要利用传感器、JSBridge、UA、安装来源、调试状态和运行环境之间的语义关系。该结果与本文系统贡献相一致: 系统建立了 Android Native、WebView 和 Web 前端之间可采集、可对齐、可解释的跨层设备指纹关系。

4.7 本章讨论与局限性

4.7.1 实验结论汇总

本章实验可以归纳为四点结论:

第一, 粗粒度三端消融显示, 单端或双端原始字段在随机划分下也能取得较低误差, 说明三端原始字段之间存在较强冗余和代理关系。

第二, 跨层一致性消融显示, 显式一致性特征可以达到与完整原始三端特征相近甚至更优的效果, Tri-layer semantic 仅 7 个特征就接近完整三端原始特征。

第三, 分组交叉验证显示, 随机 holdout 会高估泛化能力, 而核心三端语义规则在分组设置下仍优于 Raw all。

第四, 特征重要性分析显示, 模型重点依赖传感器与 JSBridge 完整性、Native 与 Web UA 匹配、WebView 宿主标记、manual 安装来源与 ADB 等跨层语义信号。

4.7.2 局限性说明

本章实验仍存在一些局限。首先, 当前数据集规模有限, 真实物理设备、厂商系统版本、WebView provider 版本和网络环境覆盖仍有扩展空间。其次, 高风险攻击样本主要来自 API 重放、无头 PC 浏览器和廉价模拟器等模板化构造, 能够验证典型风险场景, 但还不能完全代表真实攻击链路。再次, 风险标签由规则知识库驱动的大模型离线



标注流程生成,仍需要人工抽样复核或业务标注进一步校准。此外,本章主要关注离线消融实验,尚未将端侧运行耗时、功耗、内存占用和大规模真机闭环作为主要实验。最后,当前分组策略依赖启发式 `source_type` 和 `group_id` 构造,后续可在采集阶段记录更明确的分组元数据。

4.7.3 后续改进方向

后续工作可以从五个方面展开:第一,扩大真实设备覆盖,增加不同品牌、系统版本、WebView 内核版本和网络环境下的样本;第二,构造更多跨层矛盾专项样本,例如 Android 真机搭配 Windows UA、屏幕 DPR 不一致、WebGL 软件渲染、JSBridge 缺失等;第三,引入人工抽样复核,校准规则知识库和大模型标签;第四,对端侧评分 App 进行系统开销测试;第五,研究更轻量的规则执行或模型压缩方案,使三端语义规则更适合移动端部署。

4.8 本章小结

本章围绕已标注三端设备指纹数据完成了三类实验:粗粒度三端消融、一致性特征消融和分组交叉验证。结果表明,三端原始字段均包含风险信息,但直接拼接可能受到字段冗余和同源样本泄漏影响;将设备身份、UA、屏幕 DPR、WebView provider、JS Bridge、传感器、安装来源等关系显式编码后,核心三端语义规则在分组验证中表现更稳定。上述结果为 HybridGuard 的跨层一致性建模和端侧轻量评分闭环提供了实验依据。



结论

本文围绕移动端 Hybrid App 场景下的无感风控需求,设计并实现了一个基于三端融合设备指纹的原型系统 HybridGuard。系统的核心思想是:同一移动端会话中的 Android Native 原生层、WebView 宿主容器层和 Web 前端运行层共同构成跨层设备环境视图,并能够在设备身份、系统版本、屏幕参数、WebView 内核、JSBridge、传感器、WebGL 和运行环境等方面形成可对齐、可互证、可发现矛盾的关系。相比只依赖单端指纹或简单堆叠字段的方案,本文更关注三端特征之间的语义一致性和风险解释能力。

本文完成的主要工作包括以下几个方面

第一,设计并实现了三端设备指纹采集链路。系统在旧 app 模块中完成 Native、WebView 和 Web 前端的异步采集与 session_id 对齐,在新 riskapp 模块中进一步迁移为本地探针和端侧回传机制,使同一移动端会话能够形成可合并、可审计、可用于跨层分析的设备环境视图。

第二,设计并实现了后端会话合并与数据持久化流程。FastAPI 后端能够接收 Native 与 Web/WebView 的异步上报,根据 session_id 增量合并三端数据,并同时保存原始嵌套 JSON 与面向训练的 JSONL 数据。该设计兼顾了原始样本审计和后续模型训练需求。

第三,构建了面向三端融合风控的综合规则知识库。规则库覆盖核心完整性、Native-Web 一致性、Native-WebView 一致性、WebView-Web 一致性、物理运行环境、攻击场景聚合和容错规则等内容,用于约束大模型离线生成 risk_score 和 risk_reason。通过这种方式,本文将大模型的语义分析能力与可解释规则相结合,避免仅依赖少量固定阈值或完全黑盒判断。

第四,构建了风险评分数据集和轻量评分器训练流程。本文基于真实设备样本、扩充样本和高危模拟样本形成带风险标签的数据集,并使用随机森林与 MLP 作为轻量评分器候选实现。最终,本文选择随机森林作为端侧评分器的主要工程实现,并通过 m2cgen 导出 Java 推理代码。

第五,实现了 App 端侧评分闭环。新的 riskapp 模块在本地完成 Native、WebView



和 Web 三端采集, 通过 RiskFeatureEncoder 按训练阶段一致的 65 维字段顺序构造输入向量, 再调用 DeviceRiskScorer 输出风险分。端侧运行时仅上传风险分、风险等级和评分摘要, 不上传完整原始三端指纹, 体现了本地闭环和数据最小化思想。

本文基于 1323 条带风险标签的三端设备指纹样本开展消融实验和分组交叉验证。实验表明, Native、WebView 和 Web 原始字段均包含风险信息, 但随机 holdout 下的高指标受到同源样本和字段冗余影响, 简单堆叠三端原始字段并不总是最优。

跨层一致性实验进一步表明, 设备型号与 UA、Android 版本、屏幕 DPR、WebView provider、JSBridge、传感器、安装来源和调试状态等关系可以形成更贴近规则知识库判断逻辑的风险信号。尤其在分组交叉验证中, Tri-layer semantic 仅使用 7 个核心语义特征便优于完整原始三端特征, 说明稳定的跨层规则比单纯增加字段数量更有泛化价值。

特征重要性分析显示, 模型主要依赖传感器与 JSBridge 完整性、Native 与 Web UA 匹配、WebView 宿主标记、manual 安装来源与 ADB 等信号。这些结果共同支撑本文的核心结论: HybridGuard 的价值不在于简单采集更多字段, 而在于将 Android Native、WebView 宿主和 Web 前端之间的关系转化为可解释、可训练、可端侧部署的风险评分能力。

本文仍存在以下不足。首先, 当前数据集规模有限, 真实物理设备、不同厂商系统版本、不同 WebView 内核和真实攻击环境覆盖仍不充分。高风险样本主要来自模板化构造, 能够验证典型攻击场景, 但还不能完全代表真实黑产链路。

其次, 风险标签由规则知识库驱动的大模型离线标注流程生成, 虽然具备较强解释性, 但仍需要更多人工抽样复核和业务标注校准。后续可以将规则知识库进一步结构化, 加入规则版本管理、人工审核记录和冲突规则处理机制, 使标签生成过程更加稳定和可审计。

再次, 本章实验主要关注离线消融和分组泛化分析, 尚未对端侧评分 App 在大规模真实设备上的采集耗时、推理耗时、内存占用、功耗和长期稳定性进行充分评估。后续需要在更多真机和云真机环境中补充端侧闭环实验, 验证本地评分在真实部署中的工程可行性。

最后, 当前端侧评分器主要使用随机森林 Java 代码完成推理。后续可进一步研究特征筛选、模型剪枝、规则执行引擎或更轻量的模型压缩方案, 降低端侧代码体积和维



护成本。同时，后续还可扩展行为特征、网络环境特征和更细粒度 Web API 探针，使 HybridGuard 从原型系统逐步演进为更完整的移动端无感风控框架。

综上，本文完成了从三端特征采集、会话对齐、规则知识库、大模型离线标注、轻量评分训练到端侧评分闭环的完整原型验证。实验结果表明，Android Native、WebView 宿主和 Web 前端之间的跨层一致性关系能够为移动端风险识别提供有效且可解释的信号，具有进一步研究和工程完善价值。



致谢

在毕业论文即将完成之际,回顾从选题、系统实现到实验分析和论文撰写的全过程,我深切感受到这项工作并非只依靠个人完成。在此,谨向所有给予我指导、帮助和支持的老师、同学、家人和朋友致以诚挚的感谢。

首先,衷心感谢我的导师巫蜀江老师在论文选题、研究思路、系统设计和论文修改过程中给予的耐心指导。老师严谨的治学态度、清晰的问题意识和认真负责的工作作风,使我在移动端风险识别、跨层设备指纹分析以及实验论证等方面获得了更加系统的理解,也帮助我在遇到困难时不断调整方向、完善工作。

感谢学院各位任课老师在本科阶段的培养。课程学习和实践训练为本文涉及的 Android 开发、Web 技术、数据分析和机器学习方法奠定了重要基础。感谢同学们在资料查找、系统测试、论文修改和答辩准备过程中给予的交流与帮助,这些讨论让我及时发现问题,也使论文内容更加完整。

最后,感谢我的家人和朋友长期以来的理解、鼓励与支持。正是他们在学习和生活中的陪伴,让我能够以更加踏实的心态完成本次毕业设计。未来我将继续保持求实、认真和持续学习的态度,把本科阶段的积累转化为新的起点。



参考文献

- [1] Papaioannou M, Pelekoudas-Oikonomou F, Mantas G, et al. A survey on quantitative risk estimation approaches for secure and usable user authentication on smartphones[J]. *Sensors*, 2023, 23(6): 2979.
- [2] Picard C, Pierre S. RLAAuth: A risk-based authentication system using reinforcement learning[J]. *IEEE access*, 2023, 11: 61129-61143.
- [3] Pau K N, Lee V W Q, Ooi S Y, et al. The development of a data collection and browser fingerprinting system[J]. *Sensors*, 2023, 23(6): 3087.
- [4] Wu S, Sun P, Zhao Y, et al. Him of many faces: characterizing billion-scale adversarial and benign browser fingerprints on commercial websites[C]//NDSS. 2023.
- [5] Sanchez-Rola I, Bilge L, Balzarotti D, et al. Rods with laser beams: Understanding browser fingerprinting on phishing pages[C]//32nd USENIX Security Symposium (USENIX Security 23). 2023: 4157-4173.
- [6] Doty N. Mitigating browser fingerprinting in web specifications[J]. W3C, W3C Editor' s Draft, 2018.
- [7] Snyder P, Karami S, Edelstein A, et al. {Pool-Party}: Exploiting browser resource pools for web tracking[C]//32nd USENIX Security Symposium (USENIX Security 23). 2023: 7091-7105.
- [8] Annamalai M S M S, Bilogrevic I, De Cristofaro E. FP-Fed: privacy-preserving federated detection of browser fingerprinting[J]. *arXiv preprint arXiv:2311.16940*, 2023.
- [9] Sun Z, Sun R, Liu C, et al. Shadownet: A secure and efficient on-device model inference system for convolutional neural networks[C]//2023 IEEE Symposium on Security and Privacy (SP). IEEE, 2023: 1596-1612.
- [10] 刘奇旭,刘心宇,罗成,等.基于双向循环神经网络的安卓浏览器指纹识别方法[J]. *计算机研究与发展*,2020,57(11):2294-2311.
- [11] Boussaha S, Hock L, Bermejo M, et al. FP-tracer: Fine-grained browser fingerprinting detection via taint-tracking and entropy-based thresholds[J]. *Proceedings on Privacy Enhancing Technologies*, 2024.
- [12] Huyghe M, Quinton C, Rudametkin W. BrowserFM: A Feature Model-based



- Approach to Browser Fingerprint Analysis[C]//MADWeb 2025-Workshop on Measurements, Attacks, and Defenses for the Web. 2025: 1-10.
- [13]Cao Y, Li S, Wijmans E. (Cross-) browser fingerprinting via OS and hardware level features[C]//Proceedings 2017 Network and Distributed System Security Symposium. Internet Society, 2017.
- [14]Laor T, Mehanna N, Durey A, et al. Drawnapart: A device identification technique based on remote gpu fingerprinting[J]. arXiv preprint arXiv:2201.09956, 2022.
- [15]Vastel A, Laperdrix P, Rudametkin W, et al. {Fp-Scanner}: The privacy implications of browser fingerprint inconsistencies[C]//27th USENIX Security Symposium (USENIX Security 18). 2018: 135-150.
- [16]宋天乐,蔺琛皓,高书馨,等.基于移动智能终端交互行为的持续身份认证技术综述[J].信息通信技术与政策,2024,50(01):19-31.
- [17]Jiang Y, Zhu H, Chang S, et al. MAUTH: Continuous user authentication based on subtle intrinsic muscular tremors[J]. IEEE Transactions on Mobile Computing, 2023, 23(2): 1930-1941.
- [18]Shen Z, Li S, Zhao X, et al. CT-Auth: Capacitive touchscreen-based continuous authentication on smartphones[J]. IEEE Transactions on Knowledge and Data Engineering, 2023, 36(1): 90-106.
- [19]Gattulli V, Impedovo D, Pirlo G, et al. Touch events and human activities for continuous authentication via smartphone[J]. Scientific Reports, 2023, 13(1): 10515.
- [20]Stylios I, Chatzis S, Thanou O, et al. Continuous authentication with feature-level fusion of touch gestures and keystroke dynamics to solve security and usability issues[J]. Computers & Security, 2023, 132: 103363.
- [21]Heid K, Heider J. Haven't we met before?-Detecting Device Fingerprinting Activity on Android Apps[C]//Proceedings of the 2024 European Interdisciplinary Cybersecurity Conference. 2024: 11-18.
- [22]Kıymık E, Öztürk A E. Gyroscope-Based Smartphone Model Identification via WaveNet and EfficientNetV2 Ensemble[J]. IEEE Access, 2024, 12: 195483-195504.
- [23]Tiwari A, Prakash J, Gross S, et al. A large scale analysis of android—web



- hybridization[J]. Journal of Systems and Software, 2020, 170: 110775.
- [24]GOOGLE. WebView – Native bridges[EB/OL]//Android Developers. (2024)[2026-05-14]. <https://developer.android.com/privacy-and-security/risks/insecure-web-view-native-bridges>.
- [25]崔孟娇,姜政伟,陈奕任,等.面向威胁情报的大语言模型技术应用综述[J].信息安全学报,2024,9(05):1-25.DOI:10.19363/J.cnki.cn10-1380/tn.2024.09.09.
- [26]赖清楠,金建栋,周昌令.基于大语言模型的网络威胁情报知识图谱构建技术研究[J].通信学报,2024,45(S2):33-43.
- [27]GOOGLE. Build web apps in WebView[EB/OL]//Android Developers. (2024)[2026-05-14]. <https://developer.android.com/develop/ui/views/layout/webapps/webview>.
- [28]Tiwari A, Prakash J, Hammer C. Demand-driven information flow analysis of WebView in Android hybrid apps[C]//2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). IEEE, 2023: 415-426.
- [29]El-Zawawy M A, Losiouk E, Conti M. Vulnerabilities in Android webview objects: Still not the end![J]. Computers & Security, 2021, 109: 102395.
- [30]袁牧,张兰,姚云昊,等.面向智能物联网的资源高效模型推理综述[J].计算机学报, 2024,47(10):2247-2273.
- [31]Kwon C, Kang D. Overlay-ML: Unioning memory and storage space for on-device AI on mobile devices[J]. Applied Sciences, 2024, 14(7): 3022.
- [32]Li Z, Paolieri M, Golubchik L. Inference latency prediction for CNNs on heterogeneous mobile devices and ML frameworks[J]. Performance Evaluation, 2024, 165: 102429.
- [33]Abadade Y, Temouden A, Bamoumen H, et al. A comprehensive survey on tinyml[J]. IEEE access, 2023, 11: 96892-96922.
- [34]Hasanov I, Virtanen S, Hakkala A, et al. Application of large language models in cybersecurity: A systematic literature review[J]. IEEE access, 2024, 12: 176751-176778.
- [35]GOOGLE. Load in-app content[EB/OL]//Android Developers. (2022)[2026-05-16]. <https://developer.android.com/develop/ui/views/layout/webapps/load-local-content>.
- [36]Polatidis N, Kapetanakis S, Trovati M, et al. FSSDroid: Feature subset selecti



on for Android malware detection[J]. World Wide Web, 2024, 27(5): 50.